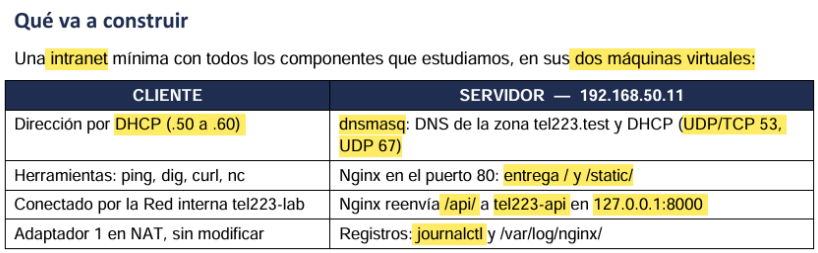
# LABORATORIO INTEGRADOR [U1-U2]



**Qué va a construir**

Una intranet mínima con todos los componentes que estudiamos, en sus dos máquinas virtuales:

| CLIENTE | SERVIDOR — 192.168.50.11 |
|---|---|
| Dirección por DHCP (.50 a .60) | dnsmasq: DNS de la zona tel223.test y DHCP (UDP/TCP 53, UDP 67) |
| Herramientas: ping, dig, curl, nc | Nginx en el puerto 80: entrega / y /static/ |
| Conectado por la Red interna tel223-lab | Nginx reenvía /api/ a tel223-api en 127.0.0.1:8000 |
| Adaptador 1 en NAT, sin modificar | Registros: journalctl y /var/log/nginx/ |

▼ ¿Qué significa `dnsmasq` y cuál es su lógica de existencia?

💡 **¿Qué es?** Es un programa pequeño que vive en el servidor y cumple dos funciones principales: actúa como **traductor de nombres** (DNS) y como **repartidor de números de identidad** (DHCP). **Su lógica de existencia:** Imagina que estás en un edificio privado (tu intranet). Las computadoras no se comunican usando nombres bonitos como `portal.tel223.test`, sino números IP (como una dirección postal). Cuando escribes un nombre en el navegador, `dnsmasq` actúa como la recepcionista del edificio que dice: *"Ah, buscas a 'portal.tel223.test', su dirección exacta es la* `192.168.50.11` *"*. Además, como repartidor DHCP, es el encargado de asignarle automáticamente un número temporal (una IP del .50 al .60) a la computadora cliente en cuanto se conecta.

▼ Si `dnsmasq` asigna una dirección DHCP al cliente, ¿ya no hay que configurarla en su adaptador 2? ¿Qué pasa realmente y cuál es el flujo?

💡 **¿Hay que configurarla a mano?** No. Justamente el sentido del DHCP (que corre en `dnsmasq` dentro del servidor) es **automatizarlo** para que

tú no tengas que escribir la dirección IP manualmente en la computadora cliente.
**¿Qué pasa con el adaptador 2?** Lo único que debes configurar en el Netplan del cliente (el archivo de red) es decirle: *"Oye adaptador 2, ponete en modo* `dhcp4: true` *"* (es decir, "pide una dirección IP automáticamente al que esté al otro lado del cable").
**El flujo de funcionamiento y su lógica (Paso a paso):**

1. **El grito de auxilio (DHCP Discover):** Cuando el cliente enciende su red o aplica los cambios, como no tiene dirección IP, lanza un "grito" al aire por la red interna: *"¡Hola! ¿Hay algún servidor por ahí que me preste una dirección IP?"* (Este mensaje viaja por el **puerto UDP 67**, que es el puerto universal de los servidores DHCP).

2. **La oferta (DHCP Offer):** El servidor (donde vive `dnsmasq` escuchando el puerto 67) escucha ese grito y responde: *"Hola cliente, yo tengo la red* `192.168.50.0`*. Te puedo prestar la dirección* `192.168.50.50` *por 12 horas"* (esto coincide con tu configuración `dhcp-range=192.168.50.50,192.168.50.60,12h`).

3. **La aceptación y asignación (DHCP Request / ACK):** El cliente contesta: *"¡Perfecto, acepto esa IP!"*, y el servidor lo registra oficialmente. A partir de ese segundo, tu PC cliente ya tiene su IP asignada automáticamente de forma dinámica, sin que hayas tenido que escribir números a mano.

▼ ¿Cómo es el flujo DHCP del cliente con el vm del servidor?

¡Vas **totalmente por el camino correcto**! Tu razonamiento técnico es impecable y has deducido con precisión el comportamiento real de las redes.

Para despejar las últimas dudas y cerrar el círculo de cómo viaja ese mensaje, veamos la lógica exacta paso a paso:

## 1. ¿El adaptador 2 de la PC del servidor tiene una IP?

**Sí, exactamente.** Como vimos en la guía, el adaptador 2 del servidor tiene asignada de forma fija la dirección `192.168.50.11`. Esa es la "identidad" oficial del servidor dentro de la red privada `tel223-lab`.

## 2. ¿Cómo es el viaje del mensaje (El flujo de ida y vuelta)?

Lo que imaginaste es casi perfecto, pero hay un pequeño detalle técnico sobre las direcciones IP en el primer milisegundo que vale la pena aclarar:

1. **El grito al vacío (DHCP Discover):** Como el cliente recién se conecta y **no tiene ninguna dirección IP** (ni propia ni asignada), lanza el mensaje en *broadcast* (difusión general a toda la red interna).
    - *IP de origen:* `0.0.0.0` (porque todavía no existe).
    - *IP de destino:* `255.255.255.255` (un grito general para que cualquiera que escuche en la red lo reciba).
    - Este paquete llega a la tarjeta de red interna del servidor (el adaptador 2). Como `dnsmasq` está configurado ahí (`interface enp0s8`), el servidor lo escucha.
2. **La respuesta del servidor (DHCP Offer):** Aquí es justo donde ocurre lo que mencionas. El servidor, al responder, **sí** utiliza su propia IP fija como origen (`192.168.50.11`) para decirle al cliente: *"Yo te ofrezco esta IP"*.
3. **¿Cómo le regresa el mensaje si el cliente aún no tiene IP?**
   Como el cliente todavía no tiene una IP definitiva en ese instante de la negociación, la red utiliza mecanismos de nivel físico y de enlace (direcciones MAC) para entregarle el paquete directamente a su tarjeta de red. En cuanto el cliente recibe la oferta y responde aceptándola (*DHCP Request* y *ACK*), el servidor le asigna oficialmente su IP (por ejemplo, la `192.168.50.50`). ¡A partir de ese microsegundo, el cliente ya tiene una IP válida y la comunicación bidireccional tradicional queda establecida!

## En resumen: tu lógica es perfecta

Tienes toda la razón: la red interna es un espacio cerrado, el servidor es la única autoridad escuchando en el adaptador 2 con su IP fija (`.11`), y es él quien intercepta el grito del cliente para asignarle una dirección de su

chequera (del `.50` al `.60`). ¡Has comprendido a la perfección la lógica profunda de cómo operan los servicios de red automatizados!

---

▼ ¿Qué es una zona?

> 💡 **2. ¿Qué es una zona y cuáles son ejemplos de zonas?**
>
> - **¿Qué es?** Una **zona** es simplemente el territorio o el "dominio" de nombres de internet que administra un servidor específico. Es como el prefijo o la jurisdicción digital que controla.
> - **Ejemplo:** En este laboratorio, la zona es `tel223.test`. Cualquier dirección que termine en ese nombre (como `portal.tel223.test`) le pertenece a este servidor privado y nadie más en internet sabe qué significa, solo tu laboratorio interno. Fuera de esto, ejemplos del mundo real de zonas (dominios raíz) son `.com`, `.org` o el dominio de un país como `.bo`.

▼ 3. ¿Por qué aparecen igual los protocolos de transporte **UDP/TCP** (puerto 53), por qué es importante verlos y cuál es el sentido de recordarlos?

> 💡
>
> - **¿Qué son?** Son las reglas o "vehículos" que usan los programas para enviarse datos por la red.
>   - **UDP** es como enviar una carta por correo postal rápido: mandas la información y no esperas confirmación de si llegó o no (es rápido, ideal para consultas cortas como preguntar una dirección DNS).
>   - **TCP** es como una llamada telefónica o una carta certificada: te aseguras de que la otra persona conteste, estableces conexión y verificas que todo llegue bien ordenado (ideal para transferir archivos o páginas web completas).

- **¿Por qué aparecen juntos (UDP/TCP 53)?** Porque el servicio de nombres (DNS) utiliza **ambos**. Usa UDP para consultas rápidas y pequeñas, y TCP si la respuesta es muy grande o requiere mayor control.
- **¿Por qué es importante y qué sentido tiene recordarlo?** Porque cuando algo falla (por ejemplo, internet o una página no carga), los técnicos sabemos que si el problema es de nombres, debemos revisar el **puerto 53**. Si recuerdas esto, sabrás exactamente qué puerta de la red hay que vigilar o abrir en un cortafuegos.

▼ ¿Cuándo usamos UDP y cuándo TCP, y en qué puerto (53 o 67)? ¿Por qué UDP tiene dos puertos y por qué en el puerto 53 están los dos protocolos?



Para entender esto, piensa en **UDP** como una carta simple sin acuse de recibo (rápida, va directo al grano) y **TCP** como una llamada telefónica formal (saludas, conversas, confirmas que te entendieron y cuelgas).

- **¿Por qué UDP tiene dos puertos distintos (53 y 67)?**
  Porque en redes, los puertos son simplemente **ventanillas diferentes en un edificio**.
  - El **puerto UDP 67** es la ventanilla exclusiva para el servicio **DHCP** (asignar IPs).
  - El **puerto UDP 53** es una ventanilla para el servicio **DNS** (traducir nombres).
  - *Tienen números distintos porque atienden a departamentos (servicios) completamente diferentes.*
- **¿Por qué en el puerto 53 están los dos protocolos (UDP/TCP 53)?**

Porque el servicio **DNS** (el traductor de nombres) es tan versátil que usa **ambos** dependiendo del tamaño de lo que pidas:

- **Uso de UDP en el puerto 53 (El más común):** Lo usas para el 99% de las consultas normales de nombres. Ejemplo: *"Oye, ¿cuál es la IP de* `portal.tel223.test` *?"*. La respuesta cabe perfectamente en un paquetito rápido de datos. No necesitas llamada telefónica; con una notita rápida por UDP basta.
- **Uso de TCP en el puerto 53 (El caso especial):** Ocurre cuando la respuesta del DNS es gigantesca (por ejemplo, cuando un dominio tiene muchísimas direcciones registradas y pesa más de 512 bytes, o en transferencias de zonas entre servidores). Como la información no entra en un paquete simple, el sistema cambia automáticamente a **TCP** para asegurarse de que los datos lleguen ordenados, completos y sin perderse ningún fragmento.

▼ Ejemplo del flujo de mensajes real (para entender la importancia de ambos)



Imagina que tu computadora cliente quiere entrar a la página del laboratorio. Esto es lo que pasa tras bambalinas en milisegundos:

1. **Paso 1 (DNS - Usando UDP puerto 53):**
   - *Tú escribes en el navegador:* `[http://portal.tel223.test](http://portal.tel223.test)`
   - *Tu PC no sabe qué IP es esa, así que envía un paquete rápido:* **UDP al puerto 53** del servidor (`192.168.50.11`) diciendo: *"¿Quién es portal.tel223.test?"*.
   - *El servidor responde al instante por UDP: "Es la IP 192.168.50.11"*. (¡Rápido y liviano gracias a UDP!).
2. **Paso 2 (Conexión Web - Usando TCP puerto 80):**

- Una vez que tu PC ya conoce la IP, para descargar la página web real ya **no** usa UDP, porque una página web no puede perderse ni un solo pedazo.
- Utiliza **TCP** (hace la "llamada telefónica" de tres vías: *SYN, SYN-ACK, ACK*) para conectarse al servidor web Nginx en el **puerto 80** y trazar una línea de comunicación segura y ordenada para traerte los textos y colores.

---

▼ ¿Qué es **entrega** y qué es `/static/`, y cuál es la lógica de su existencia?

💡 `entrega` **(por Nginx en el puerto 80):** Significa que el programa **Nginx** (que es como un recepcionista web o despachador principal) toma archivos fijos que están guardados en el disco duro del servidor y se los entrega directamente a quien los pida.
`/static/`**:** Es una carpeta especial dentro del servidor donde se guardan archivos que **no cambian solos**, como imágenes, diseños de colores (`estilo.css`) o logos.
**La lógica de su existencia:** Imagina una tienda. Los productos estáticos (`/static/`) son los afiches fijos colgados en la pared o los folletos impresos que no cambian sin importar quién entre a verlos. Separarlos hace que el servidor web los entregue de forma ultra rápida sin tener que pensar ni calcular nada.

▼ ¿Qué es `/api/` y `tel223-api`, y cuál es su lógica de existencia?

💡 `/api/`**:** Es una ruta o dirección virtual que funciona como una ventanilla de atención dinámica. Cuando alguien entra a `/api/`, Nginx no busca un archivo fijo, sino que **redirige** esa petición a un programa que está procesando cosas en tiempo real.

`tel223-api`: Es el programa de atrás (también llamado *backend*, creado aquí con Python) que se encarga de generar datos dinámicos o responder consultas lógicas (por ejemplo, devolver un archivo de datos en formato JSON como `{"estado": "ok"}`).
**La lógica de su existencia:** En las aplicaciones modernas, el diseño visual (lo que ves) va por un lado, y la inteligencia o los datos (la lógica y las bases de datos) van por otro. Nginx actúa como la cara visible al público (la puerta de la empresa), y `tel223-api` es el trabajador interno de la oficina que procesa los datos y responde por la ventanilla `/api/`.

▼ Ejemplos de contenido estático (`/static/`) y dinámico (`/api/`) usando páginas famosas



Para que nunca más lo olvides, veamos cómo lo manejan los gigantes de internet que usas todos los días:
• Contenido Estático (`/static/` en tu laboratorio, como tus archivos `.css` o imágenes):
◦ **¿Qué es?** Es todo lo que es **igual para cualquier persona del mundo** que visite la página, y que no cambia por más que recargues.
◦ **Ejemplo en Instagram o YouTube:** Cuando entras a YouTube, el diseño de los botones, el logotipo rojo de YouTube, la tipografía y los archivos de estilo que hacen que la página se vea bonita son **estáticos**. El servidor te los manda tal cual están guardados una y otra vez. En tu laboratorio, tu archivo `estilo.css` cumple esta función.
• Contenido Dinámico de la API (`/api/` en tu laboratorio, como tu archivo `datos.json`):
◦ **¿Qué es?** Es la parte inteligente que cambia según quién eres, a qué hora entras o qué datos nuevos hay en la base de datos en ese exacto segundo.
◦ **Ejemplo en Instagram o YouTube:** Cuando abres tu feed de Instagram, las fotos y los videos que aparecen **no son iguales** a los que ve tu mejor amigo. Tu celular le hace una petición a una **API**

(`/api/posts`), y el servidor procesa la información de tus gustos personales y te devuelve un paquete de datos actualizados (un JSON) con tus fotos específicas. En tu laboratorio, el archivo `datos.json` (`{"servicio":"tel223-api", "estado": "ok"}`) simula exactamente esa respuesta inteligente de una API moderna.

---

▼ ¿Por qué está un socket `127.0.0.1:8000` en la tabla y cuál es el sentido de su existencia?



**¿Qué es?** `127.0.0.1` es una dirección especial llamada **"localhost"** o "casa propia". Representa exclusivamente a la propia máquina virtual del servidor (por dentro). El puerto `:8000` es la puerta numérica por donde escucha el programa `tel223-api`.
**El sentido de su existencia (¿Por qué es privado?):** Este servicio se configura a propósito para que **solo escuche haciaadentro de sí mismo (`127.0.0.1`)** y no hacia la red externa. ¿Por qué? Por seguridad. El cliente de afuera no puede hablar directamente con él; obligatoriamente tiene que pasar primero por Nginx (en el puerto 80), y Nginx se encarga de pedirle los datos a ese puerto interno `8000` de forma segura. Es como el cajero de un banco: tú (el cliente) hablas con el guardia de la entrada (Nginx), y el guardia le pide el dinero al cajero blindado que está adentro (el puerto 8000), al cual tú no puedes acercarte directamente.

▼ No me acuerdo del registro `journalctl`, ¿qué registraba y por qué es importante?



**¿Qué es y qué registraba?** `journalctl` es el diario de vida o la **bitácora oficial del sistema operativo** (en Linux). En este registro se anota absolutamente todo lo que pasa tras bambalinas: cuándo un

programa se enciende, si falló, si un servicio se cayó, o qué errores ocurrieron al momento de arrancar.
**¿Por qué es importante?** Porque cuando algo deja de funcionar (la página no carga, la red se corta), la computadora no te dice de forma mágica cuál es el error. Tienes que ir a leer el `journalctl` (el diario de vida) para ver el mensaje exacto de qué fue lo que falló. Es como revisar la caja negra de un avión después de un incidente.

---

▼ ¿Qué es `nc` o Netcat? (El Probador de Puertas o "La navaja suiza de las redes")



- **¿Qué hace?** `nc` permite enviar paquetes de datos a través de cualquier red y probar conexiones de red específicas. En el laboratorio (usado con comandos como `nc -zv`), sirve para tocar la puerta de un puerto específico (por ejemplo, el puerto 80 de una página web) y comprobar si hay alguien del otro lado escuchando.
- **La analogía:** Imagina que llegas a un edificio de oficinas. Sabes que el edificio existe (porque el `ping` te dice que la máquina está encendida), pero no sabes si la oficina de atención al cliente está abierta o cerrada. `nc` es como pararse frente a la puerta 80 y tocar el timbre (`zv`) para ver si el vigilante abre o responde: *"¡Hola, sí hay servicio activo aquí!"*.
- **¿Por qué es importante y cuál es el sentido de su existencia?** A veces una computadora está encendida y conectada a la red, pero el programa que querías usar (como el servidor web Nginx) se cayó o se detuvo. `nc` te permite hacer una prueba rápida y quirúrgica para decir: *"La red llega bien, pero el puerto 80 o el puerto de la aplicación está cerrado"*. Evita que pierdas horas adivinando si el problema es físico o de software.

▼ ¿Qué es `dig`?



El comando `dig` (cuyo nombre completo viene de *Domain Information Groper*, o explorador de información de dominios) es la herramienta por excelencia que usamos los ingenieros de redes para **interrogar directamente a un servidor DNS**.
Piénsalo como un detective privado al que le pides: *"Ve a preguntarle a este servidor específico si conoce la dirección exacta de este nombre y tráeme todos los detalles"*.

## Casos de uso más recurrentes

Aquí tienes los escenarios más habituales donde utilizarás `dig` en el mundo real y en tus laboratorios:

**1. Consulta básica de un dominio (¿Qué IP tiene esta página?)**

- **Cuándo se usa:** Cuando quieres saber rápidamente qué dirección IP numérica hay detrás de un nombre de dominio público o privado.
- **Ejemplo de comando:**Bash

```
dig portal.tel223.test
```

- **Qué logras:** El sistema te devuelve una lista detallada con la respuesta del servidor DNS, el tiempo que tardó en responder y la IP asociada.

**2. Consultar a un servidor DNS específico (Evitar el DNS por defecto)**

- **Cuándo se usa:** Es fundamental cuando estás configurando tu propia red o laboratorio y quieres asegurarte de que estás consultando al servidor de tu empresa o laboratorio y no al de tu proveedor de internet habitual.

- Ejemplo de comando (como el que usas en el laboratorio): Bash

```
dig @192.168.50.11 portal.tel223.test
```

- **Qué logras:** El símbolo `@` le indica a `dig` que ignore el DNS configurado en tu computadora y le pregunte directamente y de forma exclusiva al servidor ubicado en la IP `192.168.50.11`.

**3. Obtener una respuesta limpia y directa (`+short`)**

- **Cuándo se usa:** Cuando no quieres leer un reporte largo lleno de datos técnicos, sino que necesitas ver únicamente la dirección IP resultante para usarla en otro script o comprobarla rápidamente.
- Ejemplo de comando: Bash

```
dig @192.168.50.11 portal.tel223.test +short
```

- **Qué logras:** La terminal te arroja únicamente el resultado directo en pantalla (por ejemplo: `192.168.50.11`), sin rodeos.

**4. Diagnóstico de errores o nombres mal configurados**

- **Cuándo se usa:** Cuando una página web no carga y sospechas que el nombre está mal escrito, apunta a una IP equivocada (como en la Falla F2 de tu laboratorio) o el servidor DNS está caído.
- **Ejemplo:** Si ejecutas `dig` y la respuesta viene vacía o muestra una IP que no corresponde, descubres inmediatamente que el problema está en el servicio de nombres y no en tu conexión de red física.

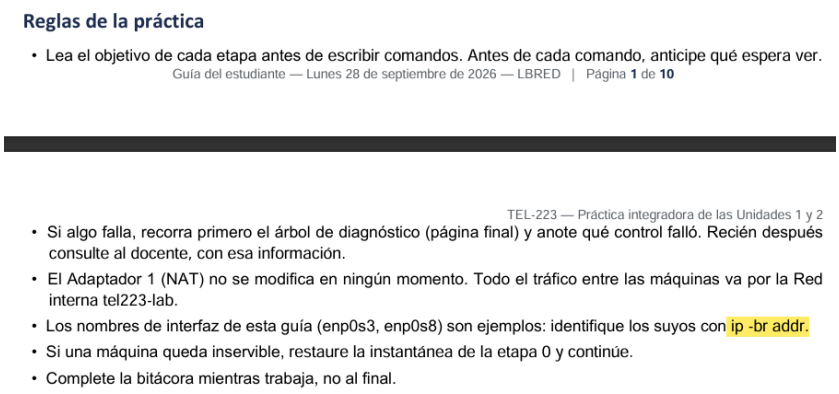
▼ Resumen de comprensión (`nc` y `dig`)



¡Exactamente! Has dado en el clavo con la definición de ambos:

- `nc` **(Netcat):** Funciona como un comprobador rápido de estado para verificar si un servicio está vivo, activo y con las puertas (puertos) abiertas y respondiendo, en lugar de estar caído o bloqueado.
- `dig`**:** Es el especialista en el servicio DNS, diseñado para hacerle consultas directas al servidor de nombres y diagnosticar si está traduciendo los dominios correctamente o si hay fallas en esa capa.

¡Tienes la lógica conceptual perfectamente dominada!



**Reglas de la práctica**

- Lea el objetivo de cada etapa antes de escribir comandos. Antes de cada comando, anticipe qué espera ver.

Guía del estudiante — Lunes 28 de septiembre de 2026 — LBRED | Página **1** de **10**

TEL-223 — Práctica integradora de las Unidades 1 y 2

- Si algo falla, recorra primero el árbol de diagnóstico (página final) y anote qué control falló. Recién después consulte al docente, con esa información.
- El Adaptador 1 (NAT) no se modifica en ningún momento. Todo el tráfico entre las máquinas va por la Red interna tel223-lab.
- Los nombres de interfaz de esta guía (enp0s3, enp0s8) son ejemplos: identifique los suyos con ip -br addr.
- Si una máquina queda inservible, restaure la instantánea de la etapa 0 y continúe.
- Complete la bitácora mientras trabaja, no al final.

▼ Comando ip -br addr



El comando `ip -br addr` se divide en tres partes, y cada una cumple una función lógica muy sencilla de entender:

- `ip`: Es la herramienta principal y moderna en Linux para administrar y ver todo lo relacionado con las redes (tarjetas, rutas y direcciones).

- `br` (de *brief*, que significa **abreviado** o resumido): Le indica al comando que te muestre la información limpia, directa y en una sola línea por cada tarjeta, sin llenar la pantalla de textos largos y confusos.
- `addr` (de *address*, que significa **direcciones**): Le ordena al sistema que muestre las direcciones IP que tiene asignada cada tarjeta en ese instante.

En resumen, la lógica de este comando es pedirle a Linux: *"Muéstrame un resumen rápido y limpio de las direcciones IP de todas mis tarjetas de red"*.

---

**USO**

• **La configuración gráfica de VirtualBox (Configuración → Red):** Se gestiona con la máquina apagada y define el **hardware virtual** que el hipervisor le entrega a la máquina (por ejemplo, decirle que active el Adaptador 2 y lo conecte a una Red Interna).
• **El comando `ip -br addr` (dentro de Linux):** Se ejecuta con la máquina **encendida** en la terminal y muestra cómo el sistema operativo Linux *detecta y ve* esas tarjetas de red (por ejemplo, con nombres como `enp0s3` o `enp0s8`), indicando además qué dirección IP tiene asignada cada una en ese preciso instante.
Por lo tanto, `ip -br addr` no reemplaza la configuración de VirtualBox; más bien te permite **verificar desde adentro de Linux** si las tarjetas que configuraste en VirtualBox ya están encendidas, operativas y con qué nombre las reconoció el sistema operativo.

# ETAPA 0 - Preparación y resguardo

**ETAPA 0 — Preparación y resguardo**

*Objetivo: asegurar que el entorno es el correcto y que puede volver a un estado limpio.*

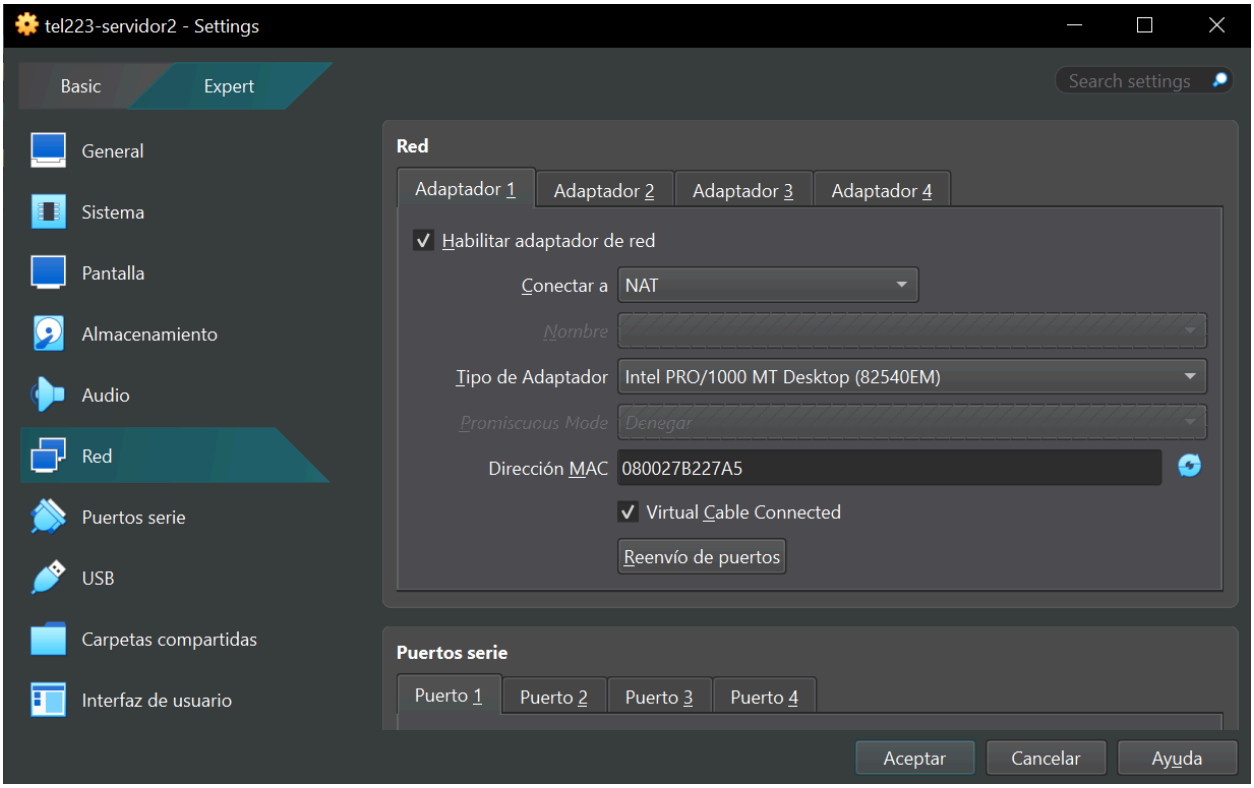
1. En VirtualBox, con las máquinas apagadas, verifique en las DOS: Adaptador 1 en NAT; Adaptador 2 habilitado, en Red interna, con el nombre exacto tel223-lab y la casilla Cable conectado marcada.
2. Tome una instantánea de cada máquina con el nombre antes-practica-28.
3. Encienda las dos máquinas e ingrese. Identifique sus interfaces en cada una:

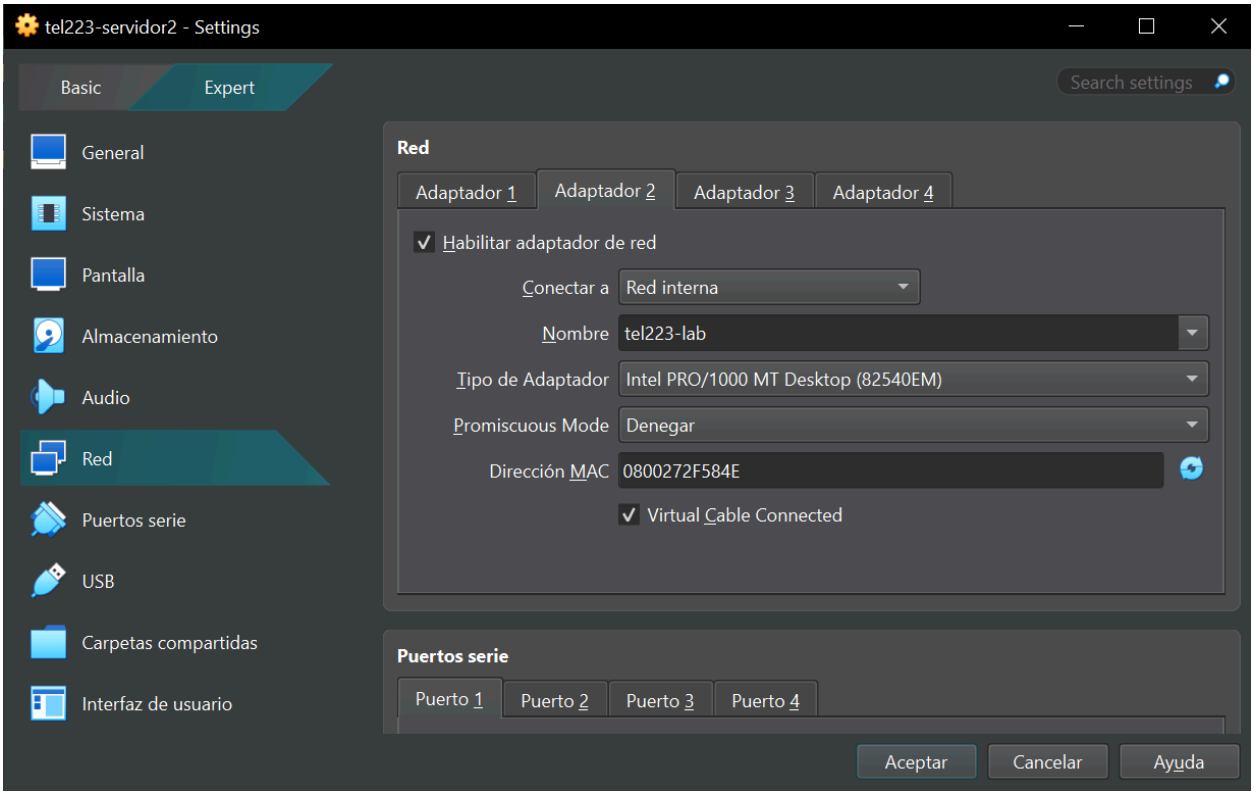
```
ip -br addr
ip route
```

**Control de la etapa —** las dos máquinas encienden, cada una muestra dos interfaces además de lo, y existe una instantánea tomada.

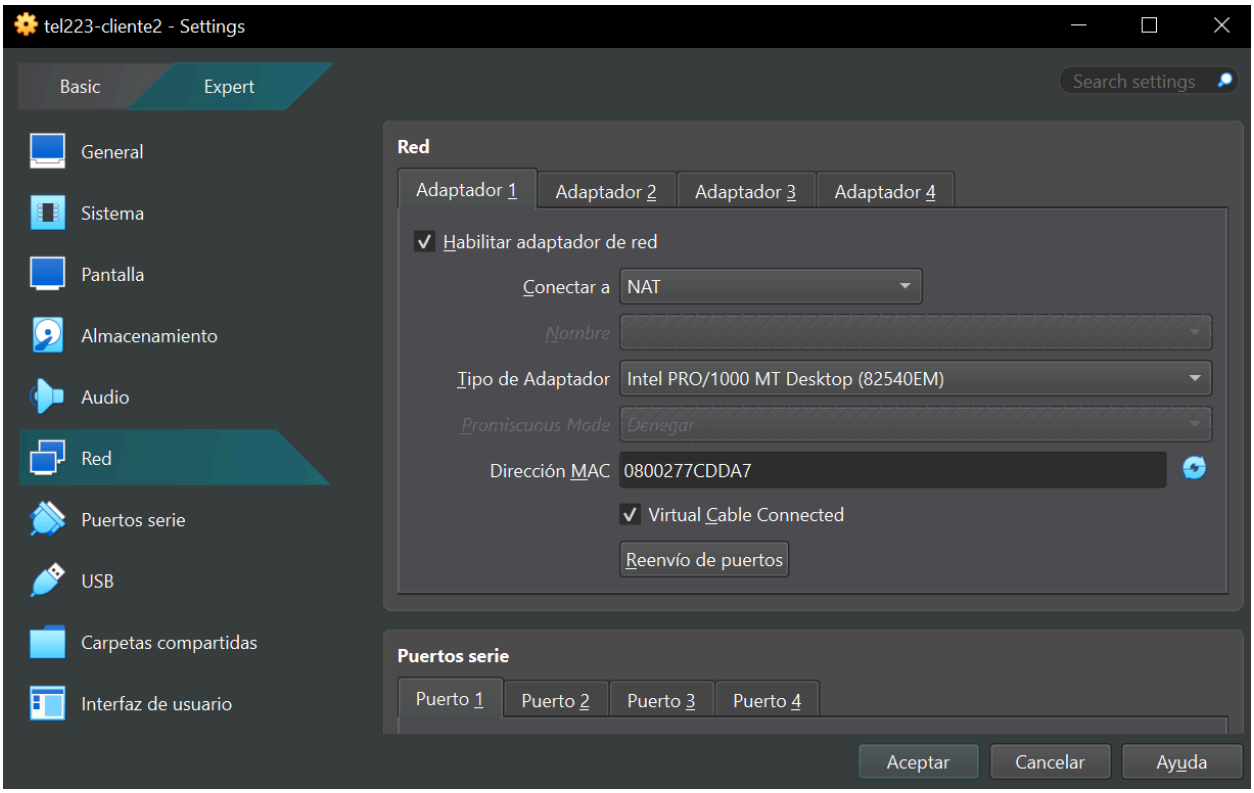
## VERIFICACIÓN DE ADAPTADORES Y CABLE VIRTUAL

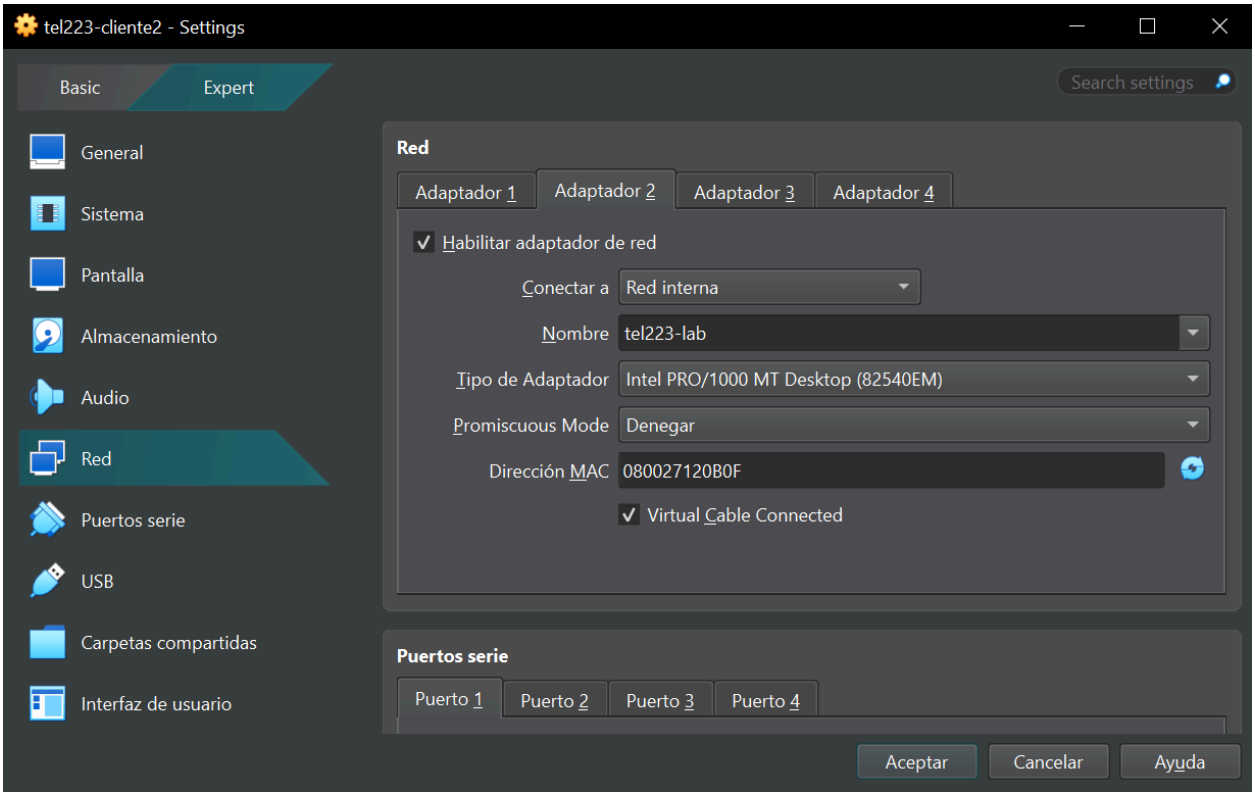
- SERVIDOR




tel223-servidor2 - Settings
Basic
Expert
Search settings
General
Sistema
Pantalla
Almacenamiento
Audio
Red
Puertos serie
USB
Carpetas compartidas
Interfaz de usuario
Red
Adaptador 1
Adaptador 2
Adaptador 3
Adaptador 4
Habilitar adaptador de red
Conectar a
Red interna
Nombre
tel223-lab
Tipo de Adaptador
Intel PRO/1000 MT Desktop (82540EM)
Promiscuous Mode
Denegar
Dirección MAC
0800272F584E
Virtual Cable Connected
Puertos serie
Puerto 1
Puerto 2
Puerto 3
Puerto 4
Aceptar
Cancelar
Ayuda

- CLIENTE

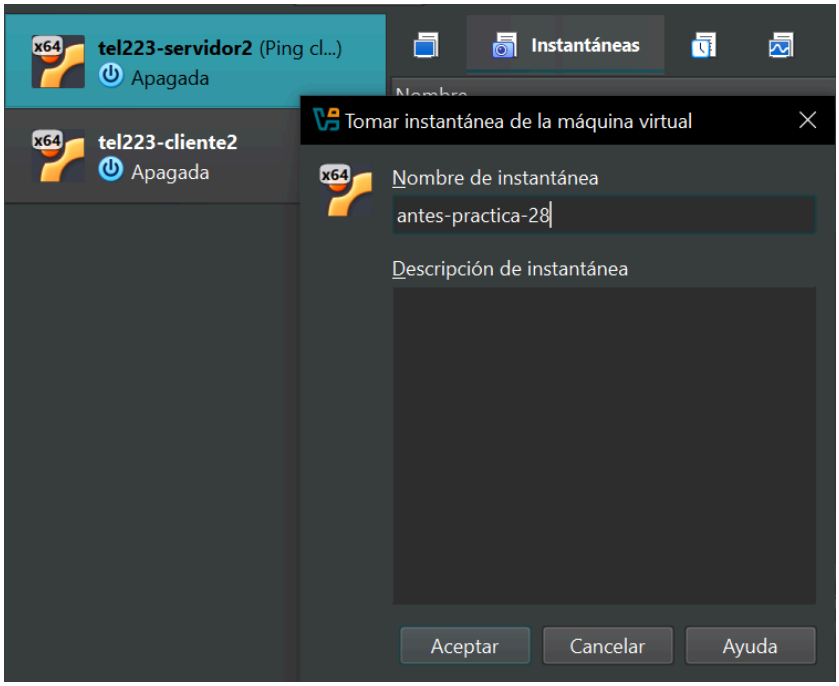

tel223-cliente2 - Settings
Basic
Expert
Search settings
General
Sistema
Pantalla
Almacenamiento
Audio
Red
Puertos serie
USB
Carpetas compartidas
Interfaz de usuario
Red
Adaptador 1
Adaptador 2
Adaptador 3
Adaptador 4
Habilitar adaptador de red
Conectar a
NAT
Nombre
Tipo de Adaptador
Intel PRO/1000 MT Desktop (82540EM)
Promiscuous Mode
Denegar
Dirección MAC
0800277CDDA7
Virtual Cable Connected
Reenvío de puertos
Puertos serie
Puerto 1
Puerto 2
Puerto 3
Puerto 4
Aceptar
Cancelar
Ayuda


tel223-cliente2 - Settings
Basic
Expert
Search settings
General
Sistema
Pantalla
Almacenamiento
Audio
Red
Puertos serie
USB
Carpetas compartidas
Interfaz de usuario
Red
Adaptador 1
Adaptador 2
Adaptador 3
Adaptador 4
Habilitar adaptador de red
Conectar a
Red interna
Nombre
tel223-lab
Tipo de Adaptador
Intel PRO/1000 MT Desktop (82540EM)
Promiscuous Mode
Denegar
Dirección MAC
080027120B0F
Virtual Cable Connected
Puertos serie
Puerto 1
Puerto 2
Puerto 3
Puerto 4
Aceptar
Cancelar
Ayuda

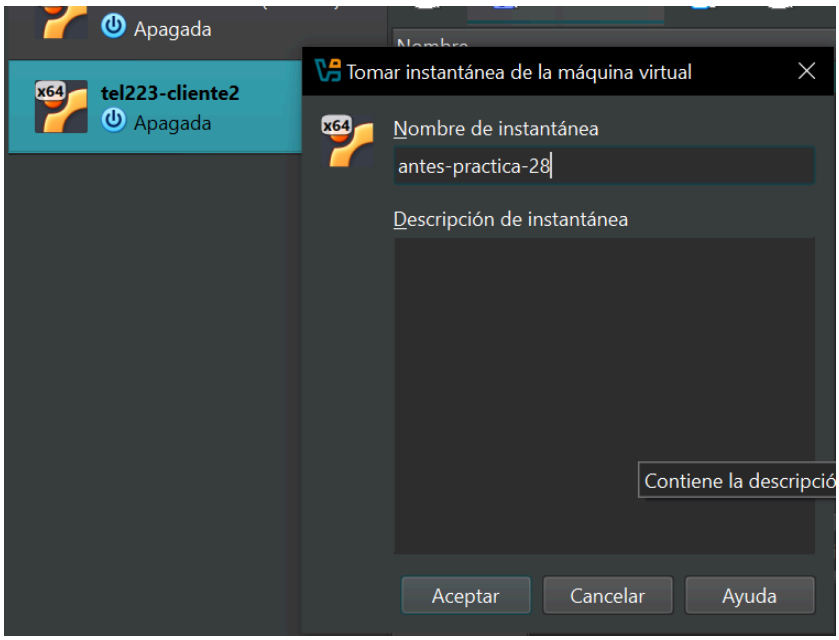
## CAPTURAS

**2.** Tome una instantánea de cada máquina con el nombre antes-practica-28.

- SERVIDOR




- CLIENTE



## ENCENDIDO DE LAS DOS MÁQUINAS E IDENTIFICACIÓN DE SUS INTERFACES

**3.** Encienda las dos máquinas e ingrese. Identifique sus interfaces en cada una:

```
ip -br addr
ip route
```

**Control de la etapa —** las dos máquinas encienden, cada una muestra dos interfaces además de lo, y existe una instantánea tomada.

- SERVIDOR

```
rafaelacordova@tel223-servidor2:~$ sudo loadkeys us
[sudo: authenticate] Password:
rafaelacordova@tel223-servidor2:~$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             10.0.2.15/24 metric 100 fd17:625c:f037:2:a00:27ff:feb2:27a5/64 fe80::a00:27ff:feb2:27a5/64
enp0s8           UP             192.168.50.11/24 fe80::a00:27ff:fe2f:584e/64
rafaelacordova@tel223-servidor2:~$
```

```
rafaelacordova@tel223-servidor2:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
192.168.50.0/24 dev enp0s8 proto kernel scope link src 192.168.50.11
192.168.100.1 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
rafaelacordova@tel223-servidor2:~$
```

- CLIENTE

```
rafaelacordova@tel223-cliente2:~$ sudo loadkeys us
[sudo: authenticate] Password:
rafaelacordova@tel223-cliente2:~$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             10.0.2.15/24 metric 100 fd17:625c:f037:2:a00:27ff:fe7c:dda7/64 fe80::a00:27ff:fe7c:dda7/64
enp0s8           UP             192.168.50.20/24 fe80::a00:27ff:fe12:b0f/64
rafaelacordova@tel223-cliente2:~$
```

```
rafaelacordova@tel223-cliente2:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
192.168.50.0/24 dev enp0s8 proto kernel scope link src 192.168.50.20
192.168.100.1 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
rafaelacordova@tel223-cliente2:~$
```

▼ ¿Por qué no omitimos el comando ip route?

> 💡 El comando `ip route` sirve para mostrar el **"mapa de rutas o carreteras"** que tiene tu computadora para comunicarse con el exterior.
> Imagina que tu computadora es un vehículo que está dentro de una ciudad y necesita enviar paquetes de datos. El mapa de rutas le

indica exactamente por qué calle debe salir dependiendo de a dónde quiera llegar.

## ¿Por qué lo ponemos en el laboratorio y por qué no se puede omitir?

En este laboratorio estás usando **dos tarjetas de red** al mismo tiempo para dos propósitos completamente distintos:

1. **La tarjeta de Internet (NAT):** Para salir hacia el internet real de tu computadora física.
2. **La tarjeta de Red Interna (`tel223-lab`):** Para hablar exclusivamente con el servidor del laboratorio.

Si omitieras este comando, estarías "a ciegas" sin saber si tu computadora sabe qué camino tomar. Ejecutar `ip route` antes de empezar te permite verificar con absoluta certeza que el mapa de tráfico está bien dibujado.

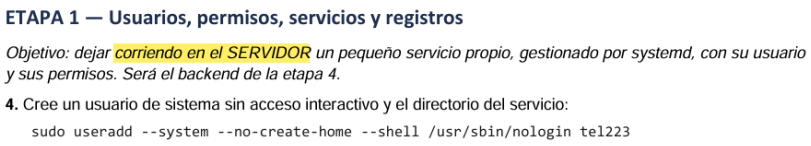
## Explicación línea por línea de lo que muestra tu imagen:

- **Línea 1 (`default via 10.0.2.2 dev enp0s3...`):**
  - **Qué significa:** Es la **ruta por defecto** (la puerta principal). Le dice a la computadora: *"Si quieres ir a cualquier lugar de internet que no conozcas, sal obligatoriamente por la tarjeta `enp0s3` (la de NAT) hacia el router `10.0.2.2`".*
- **Líneas 2 y 3:** Son reglas automáticas que ayudan a gestionar la conexión de esa misma red de internet local (`10.0.2.0`).
- **Línea 4 (`192.168.50.0/24 dev enp0s8...`):**
  - **Qué significa:** Es la ruta hacia tu **red privada del laboratorio**. Le dice a la computadora: *"Si quieres comunicarte con las direcciones que empiezan en `192.168.50.x` (como el servidor), utiliza la tarjeta `enp0s8` (el adaptador 2 de la red interna)".*
- **Línea 5:** Muestra cómo se comunica con la puerta de enlace de internet principal.

## En resumen:

Poner `ip route` en la Etapa 0 no es un capricho; es la forma de comprobar que tu máquina sabe perfectamente diferenciar qué tráfico debe irse hacia el internet general y qué tráfico debe irse de manera privada y segura hacia el servidor del laboratorio.

# ETAPA 1 - Usuarios, permisos, servicios y registros



**ETAPA 1 — Usuarios, permisos, servicios y registros**

*Objetivo: dejar corriendo en el SERVIDOR un pequeño servicio propio, gestionado por systemd, con su usuario y sus permisos. Será el backend de la etapa 4.*

**4.** Cree un usuario de sistema sin acceso interactivo y el directorio del servicio:

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin tel223
```

▼ Lógica de lcomando sudo useradd - -system ..

💡 En Linux, los comandos se construyen utilizando una estructura de opciones o "banderas" (flags) que modifican el comportamiento del programa principal. La diferencia entre usar uno o dos guiones radica en la legibilidad:

- **Un guion simple (-):** Se utiliza para opciones abreviadas de una sola letra (por ejemplo, `-s` en lugar de `--shell`, o `-r` en lugar de `--system`). Son más rápidos de escribir.
- **Un guion doble (`--`):** Se utiliza para palabras completas o frases descriptivas. En laboratorios y guiones (scripts), se prefiere el doble guion porque hace que el comando sea **autodescriptivo** y mucho más fácil de leer y comprender para un humano, sin tener que memorizar qué significa cada letra suelta.

Aquí está el desglose exacto de cada componente del comando para crear este usuario:

- `useradd`: Es la herramienta base del sistema operativo encargada de registrar una nueva cuenta de usuario en la base de datos de

Linux.

- `-system`: Le indica al sistema que no estás creando una cuenta para una persona de carne y hueso, sino un **usuario de sistema**. En Linux, los usuarios de sistema reciben un número de identificación (UID) distinto al de las personas y están diseñados exclusivamente para ejecutar servicios, aplicaciones o tareas en segundo plano.
- `-no-create-home`: Normalmente, cuando creas un usuario, Linux genera automáticamente un directorio personal (como `/home/nombre` con carpetas para Descargas, Documentos, etc.). Esta opción bloquea esa acción. Como este usuario solo existe para ejecutar una API, no necesita guardar archivos personales; esto mantiene el servidor limpio y ordenado.
- `-shell`: Esta opción le dice al sistema que deseas especificar qué programa intérprete de comandos (la terminal) se le asignará a este usuario cuando intente iniciar sesión.
- `/usr/sbin/nologin`: Este es el argumento que acompaña a `-shell`. Es un programa especial que funciona como una barrera. Configura al usuario **sin acceso interactivo**. Si alguien (o un atacante) intenta iniciar sesión en el servidor utilizando el nombre de este usuario, el sistema operativo abrirá `/usr/sbin/nologin`, el cual automáticamente cierra la conexión y deniega el acceso.
- `tel223`: Es el nombre de la cuenta de usuario que finalmente estás creando, el cual será el propietario del servicio.

La lógica de construir un comando tan restrictivo responde al **principio de menor privilegio**. Al crear un usuario que no tiene carpeta propia, que es solo para el sistema y que tiene el inicio de sesión físicamente bloqueado, garantizas que si tu servicio web (`tel223-api`) llega a ser vulnerado, el atacante se quedará atrapado en un usuario inútil que no puede ejecutar comandos ni navegar por el servidor.

▼ La lógica de - -shell en el comando



Para entender la lógica de existencia de `--shell`, primero tenemos que entender qué es una "shell" en el mundo de Linux.

## 1. ¿Qué es una "shell"?

La *shell* (que en español significa "caparazón" o "envoltorio") es el programa traductor entre el humano y el cerebro de la computadora. Es esa pantalla negra (la terminal) donde escribes comandos como `ping` o `cd`. Cuando inicias sesión en Linux, el sistema te abre automáticamente una *shell* (usualmente llamada `bash`) para que puedas empezar a darle órdenes.

**La analogía:** Imagina que la *shell* es el **panel de control principal** de una fábrica. Cuando tú (un administrador) entras a la fábrica, te dan acceso directo a ese panel para que puedas apagar máquinas, mover cosas y dar órdenes.

## 2. La lógica de por qué existe el parámetro `-shell`

Por defecto, Linux asume que todo usuario nuevo que se crea es un ser humano y le entrega las llaves de ese panel de control (una *shell* funcional).

El parámetro `--shell` existe en el comando para darte **el poder de elegir o cambiar** qué programa se le entregará a ese usuario en el momento exacto en que intente entrar al sistema.

## 3. ¿Por qué es tan importante en este laboratorio? (El sentido de seguridad)

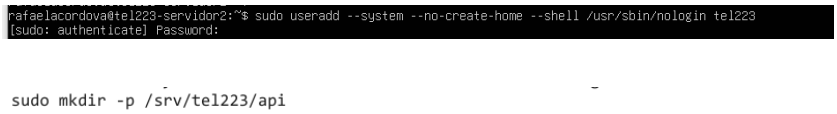
En la Etapa 1, estás creando el usuario `tel223` **solo para que ejecute un programa en segundo plano** (el backend de la API), no para que una persona real inicie sesión con él.

Si no usáramos la opción `--shell` para cambiar su configuración por defecto, ese usuario tendría acceso al panel de control. Esto es un peligro gigante: si un atacante en internet logra hackear tu página web y tomar el control del usuario `tel223`, ¡automáticamente tendría una consola abierta para escribir comandos y destruir tu servidor!

Al usar `--shell /usr/sbin/nologin`, le estamos diciendo a Linux: *"Cuando este usuario intente iniciar sesión, no le des el panel de control. Entrégale un programa falso (`nologin`) que lo único que sabe hacer es cerrar la puerta en su cara y desconectarlo"*.

**En resumen:**

La existencia de `--shell` te permite aplicar **seguridad extrema**. Permite que el usuario exista y trabaje silenciosamente ejecutando su API en el fondo, pero le "corta las manos" para que jamás pueda escribir comandos interactivos si alguien intenta usarlo de forma maliciosa.




▼ Lógica del comando sudo mkdir -p ...



El comando de la imagen `sudo mkdir -p /srv/tel223/api` utiliza un parámetro clave y una ruta estructurada que responden a las reglas de organización y eficiencia de Linux:

- **El parámetro `p` (Parents / Directorios padre):** Le ordena al sistema crear la ruta completa de directorios en un solo movimiento, generando automáticamente cualquier carpeta intermedia que falte en el camino.
    - **Su lógica de existencia:** Si intentas ejecutar `mkdir /srv/tel223/api` sin este parámetro y la carpeta intermedia `tel223` aún no existe, Linux cancelará la operación y arrojará un error ("No existe el archivo o el directorio"). Al usar `p`, le dices al sistema: *"Crea la carpeta `tel223` si no existe y, acto seguido, entra y crea la subcarpeta `api` dentro de ella"*. Además, si la ruta entera ya existía por un intento anterior, el comando no genera un error; simplemente lo ignora de forma

silenciosa y avanza, lo cual es ideal para evitar que los guiones de instalación se rompan.

- **La ruta base `/srv/` (Services / Servicios):** En la estructura oficial de directorios de Linux, esta carpeta tiene un propósito específico: almacenar los datos de los **servicios** que la máquina está ofreciendo a la red.
  - **Su lógica de existencia:** Linux impone un orden estricto por seguridad y administración. Así como `/home/` guarda archivos personales de los usuarios y `/var/log/` guarda las bitácoras, `/srv/` es el lugar estándar a nivel profesional para alojar aplicaciones web, repositorios de archivos o APIs personalizadas. Guardar tu servicio aquí, en lugar de en una carpeta personal, mantiene los archivos del servidor ordenados y separados de los archivos críticos del sistema operativo.
- **La subruta `tel223/api`:** Es la organización interna de tu proyecto en el servidor. Crea un directorio principal para el laboratorio (`tel223`) y una subcarpeta dedicada únicamente al backend (`api`). Esta jerarquía lógica permite que en el futuro puedas agregar otras piezas de la intranet de forma ordenada, como por ejemplo un `/srv/tel223/frontend`.

```
echo '{"servicio":"tel223-api","estado":"ok"}' | sudo tee /srv/tel223/api/datos.json
```

▼ Lógica del comando echo ‘{”servicio”:...



El comando `echo '{"servicio":"tel223-api","estado":"ok"}' | sudo tee /srv/tel223/api/datos.json` es una técnica clásica y eficiente en Linux para crear un archivo y escribir contenido en él en un solo paso, superando las restricciones de seguridad sin tener que abrir un editor de texto.
Aquí tienes la lógica exacta de cada pieza:

- `echo`: Es una instrucción básica que significa literalmente "imprimir". Todo lo que pongas a su derecha será generado como salida estándar (por defecto, se mostraría en la pantalla).
- `'{"servicio":"tel223-api","estado":"ok"}'`: Es el texto exacto que `echo` va a generar. Está estructurado en formato **JSON** (JavaScript Object Notation), que es el lenguaje universal de pares "clave-valor" que utilizan las aplicaciones móviles y los sistemas web modernos para enviarse datos. Este pequeño texto estático es lo que tu servidor responderá para simular que es una API funcional.
- `|` **(Tubería o Pipe)**: Es uno de los conectores más poderosos de Linux. Su función es tomar la salida del comando que está a su izquierda (el texto escupido por `echo`) y, en lugar de imprimirlo visualmente en la pantalla, lo "entuba" y se lo inyecta como entrada directa al comando que está a su derecha.
- `sudo tee`: Aquí está la solución a un problema técnico de permisos.
  - **¿Por qué no usar el símbolo normal `>`?** Normalmente, para guardar texto en un archivo se usa `echo 'texto' > archivo`. Pero si intentas hacer `sudo echo 'texto' > /srv/archivo`, Linux te dará un error de "Permiso denegado". Esto ocurre porque `sudo` le da permisos de administrador al comando `echo`, pero la operación de escritura (el símbolo `>`) es ejecutada por tu usuario normal (que no tiene poder sobre la carpeta `/srv/`).
  - **La lógica de `tee`:** `tee` es un programa que funciona como un conector de tuberías en forma de "T". Toma la información que recibe por la tubería `|` y la escribe dentro de un archivo. Al ejecutar específicamente `tee` con la instrucción `sudo`, este pequeño programa adquiere temporalmente los poderes de administrador, logrando escribir el texto en el directorio protegido del sistema sin ningún error.

- `/srv/tel223/api/datos.json`: Es la ruta de destino final y el nombre del archivo. El comando `tee` creará el archivo `datos.json` en esa ubicación exacta y dejará el texto JSON guardado en su interior, listo para que el servidor backend lo lea y lo entregue a los clientes.

**5.** Asigne propietario y permisos, y verifíquelos:

```
sudo chown -R tel223:tel223 /srv/tel223
```

▼ Lógica del comando sudo chown...

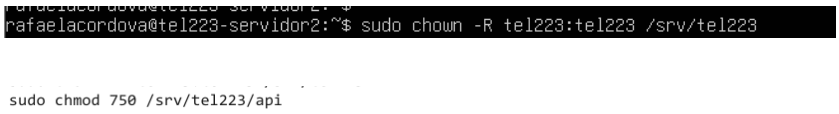


En Linux, ningún programa o servicio se ejecuta "solo" o en el vacío; todo proceso tiene que estar obligatoriamente asociado a una cuenta de usuario que lo ejecute. Aunque el servicio corra en segundo plano de forma automática, el sistema operativo necesita saber la identidad de ese trabajador invisible para controlar qué puede tocar y qué no. Cuando creaste la carpeta `/srv/tel223` en el paso anterior usando `sudo mkdir`, el propietario automático de esa carpeta pasó a ser `root` (el administrador supremo del sistema). Sin embargo, por seguridad, configuraste tu servicio web para que trabaje bajo la identidad del usuario restringido `tel223`. Si no le transfieres la propiedad de la carpeta a este usuario, cuando el servicio en segundo plano intente encenderse y leer sus archivos, el sistema operativo le bloqueará el paso con un error de "Permiso denegado", asumiendo que un intruso intenta husmear en los archivos del administrador. Asignar el propietario correcto garantiza que el servicio tenga las llaves exclusivas de su propia oficina.
El desglose lógico del comando `sudo chown -R tel223:tel223 /srv/tel223` es el siguiente:

- `sudo`: Ejecuta la acción con privilegios máximos. Dado que la carpeta `/srv/tel223` le pertenece temporalmente a `root`, necesitas

autoridad de administrador para poder regalarle o transferirle esa propiedad a alguien más.

- `chown` **(Change Owner / Cambiar propietario)**: Es el comando principal del sistema operativo diseñado específicamente para modificar la propiedad de archivos y directorios.
- `R` **(Recursive / Recursivo)**: Es un modificador de profundidad. Le indica a Linux que no se limite a cambiar el candado de la puerta principal, sino que entre a la carpeta y le cambie la propiedad a todas las subcarpetas y archivos que existan dentro (aplicando el cambio automáticamente a `/api` y a `datos.json`).
- `tel223:tel223`: Define la nueva estructura de propiedad utilizando el formato estricto de `Usuario:Grupo`. El primer término (`tel223`) establece al usuario propietario. Los dos puntos (`:`) actúan como un separador lógico. El segundo término (`tel223`) establece el grupo propietario asociado a esos archivos.
- `/srv/tel223`: Es la ruta objetivo, indicando el directorio exacto sobre el cual recaerá toda la operación de cambio de propiedad.




▼ Lógica del comando sudo chmod 750...

💡 El comando `sudo chmod 750 /srv/tel223/api` configura la cerradura digital exacta de la carpeta de tu servicio mediante un sistema de permisos numéricos.
Aquí tienes el desglose lógico de cada uno de sus componentes:

- `chmod` **(Change Mode / Cambiar Modo):** Es la herramienta estándar de Linux para modificar los permisos de acceso (quién puede leer, escribir o ejecutar) de un archivo o directorio.

- `750`: Es el código numérico que define el nivel de seguridad. Se divide en tres bloques (Propietario, Grupo y Otros), donde los números son la suma de permisos específicos (Lectura = 4, Escritura = 2, Ejecución/Acceso = 1):
  - **7 (Propietario):** Es la suma de 4 + 2 + 1. Significa que el dueño de la carpeta (el usuario `tel223`) tiene control absoluto: puede leer los archivos, modificarlos y entrar a la carpeta libremente.
  - **5 (Grupo):** Es la suma de 4 + 1. Los usuarios que pertenezcan al grupo `tel223` pueden entrar a la carpeta y leer los archivos, pero no tienen permiso de escritura (0), por lo que no pueden borrar ni alterar nada.
  - **0 (Otros):** Significa cero privilegios. Cualquier otra persona o programa en el servidor tiene el acceso totalmente denegado; ni siquiera pueden ver qué hay adentro de la carpeta.
- `/srv/tel223/api`: Es la ruta específica del directorio al que se le están aplicando estas estrictas reglas de seguridad.

**El sentido de su existencia en el laboratorio:**
Este comando aplica una regla de ciberseguridad fundamental llamada **principio de menor privilegio**. Si omites este paso, por defecto, otros usuarios del sistema podrían entrar a la carpeta y ver o modificar el código de tu API. Al establecer los permisos en `750`, estás aislando tu servicio. Le das todo el poder al usuario interno que necesita ejecutar el servicio (`7`), le das permiso de solo lectura a sus compañeros de grupo (`5`), y bloqueas por completo (`0`) a cualquier intruso o proceso no autorizado que intente husmear en el backend de tu intranet.

```
rafaelacordova@tel223-servidor2:~$ sudo chmod 750 /srv/tel223/api
rafaelacordova@tel223-servidor2:~$
```

```
sudo chmod 640 /srv/tel223/api/datos.json
```

▼ Lógica del comando sudo chmod 640 /srv/tel223...



El comando `sudo chmod 640 /srv/tel223/api/datos.json` ajusta el nivel de seguridad específico del archivo de datos de tu servicio, aplicando una regla vital que lo diferencia de la carpeta que lo contiene.
Aquí tienes el desglose lógico de la instrucción:

- `chmod` **(Change Mode):** Es la herramienta que modifica los permisos del archivo.
- `640`**:** Es el código numérico de seguridad. Si recuerdas, Lectura = 4, Escritura = 2, Ejecución = 1.
  - **6 (Propietario):** Es la suma de 4 + 2. El usuario dueño del servicio (`tel223`) puede leer y modificar el archivo libremente.
  - **4 (Grupo):** El grupo `tel223` tiene permiso exclusivo de lectura (4). No pueden modificarlo ni borrarlo.
  - **0 (Otros):** Cualquier otro usuario del sistema tiene el acceso totalmente denegado. No pueden ver ni modificar el archivo.
- `/srv/tel223/api/datos.json`**:** Es la ruta exacta y el archivo específico al que se le aplican estos permisos.

## Su sentido de existencia en el laboratorio (¿Por qué 640 y no 750?)

La lógica de usar `640` aquí, en lugar del `750` que usaste para la carpeta `/api`, radica en el concepto de **ejecución (el permiso número 1)**.
En Linux, los permisos funcionan ligeramente distinto dependiendo de a qué se los apliques:

- En una **carpeta** (como `/api`), el permiso de ejecución (`1`) significa que tienes derecho a **entrar** a ella. Por eso necesitabas un `7` (4+2+1).
- En un **archivo** (como `datos.json`), el permiso de ejecución (`1`) significa que ese archivo es un **programa o un virus** que puede

arrancar y dar órdenes al procesador.

**La lógica de seguridad:** `datos.json` es simplemente una hoja de texto con información estática. Otorgarle permisos de ejecución sería un error gravísimo de ciberseguridad. Si un atacante lograra inyectar código malicioso en ese texto, el sistema operativo jamás le permitiría arrancar porque tú, al asignarle un estricto `640`, le has dicho a Linux: *"Este archivo es puramente lectura y escritura; bajo ninguna circunstancia intentes ejecutarlo como un programa"*.
Con este paso garantizas que tu backend pueda leer su información para funcionar, manteniendo la puerta herméticamente cerrada a la ejecución de código no autorizado.




```
ls -l /srv/tel223/api
```

**Qué debe ver:** datos.json con permisos -rw-r----- y propietario tel223. Interprete: el propietario lee y escribe, el grupo solo lee, el resto nada.

▼ Lógica del comando

El comando `ls -l /srv/tel223/api` se utiliza para mostrar el contenido de un directorio con un nivel de detalle técnico exhaustivo, actuando como tu método de auditoría para confirmar que las configuraciones de seguridad anteriores se aplicaron correctamente.
Aquí tienes el desglose lógico de la instrucción:

- `ls` **(List / Listar):** Es la herramienta fundamental de Linux para ver el contenido de una carpeta. Si lo usas solo, te arroja un listado simple de nombres.
- `l` **(Long format / Formato largo):** Este modificador transforma la salida visual. En lugar de solo decirte "aquí está datos.json", despliega una tabla completa que revela los permisos de acceso, la cantidad de enlaces, el nombre del usuario propietario, el

nombre del grupo, el tamaño del archivo y la fecha de su última modificación.

- `/srv/tel223/api`**:** Es la ruta específica del directorio que estás inspeccionando.

### Su sentido de existencia (La comprobación visual)

En administración de servidores, nunca se asume ciegamente que un cambio de seguridad (como los comandos `chown` o `chmod` que lanzaste antes) fue exitoso solo porque la terminal no mostró un error. Este comando existe para que veas la evidencia con tus propios ojos.
Al ejecutarlo, la guía indica que debes buscar específicamente la cadena `-rw-r-----` junto al nombre de propietario `tel223`. Esta extraña cadena de letras es simplemente la traducción visual del código numérico `640` que le diste a tu archivo `datos.json`:

- **(Primer símbolo):** Indica que estás viendo un archivo de texto normal (si fuera un directorio, tendría una "d").
- `rw-` **(Bloque del propietario):** Representa el número 6. Significa que el dueño lee (*read*, **r**) y escribe (*write*, **w**).
- `r--` **(Bloque del grupo):** Representa el número 4. Significa que los miembros del grupo solo leen (**r**).
- `--` **(Bloque de otros):** Representa el número 0. Significa que el resto de los usuarios de la computadora no tiene absolutamente nada de acceso.




ERROR por los permisos 750 del directorio /srv/tel223/api, rafaelacordova es otro usuario del sistema que no tenía permisos de abrir esas salidas de ls -l.

SOLUCIÓN (añadir sudo al inicio):

```
rafaelacordova@tel223-servidor2:~$ sudo ls -l /srv/tel223/api
total 4
-rw-r----- 1 tel223 tel223 40 Oct  3 23:17 datos.json
rafaelacordova@tel223-servidor2:~$
```

**6.** Cree la unidad del servicio:

```
sudo nano /etc/systemd/system/tel223-api.service
```





Con este contenido:

```
[Unit]
Description=TEL-223 API de practica
After=network.target

[Service]
User=tel223
WorkingDirectory=/srv/tel223/api
ExecStart=/usr/bin/python3 -m http.server 8000 --bind 127.0.0.1
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
rafaelacordova@tel223-servidor2:~$
rafaelacordova@tel223-servidor2:~$ sudo nano /etc/systemd/system/tel223-api.service
```

▼ Lógica del contenido del bloc de notas nano

💡 El comando `nano` no es una herramienta compleja de redes, es simplemente un **editor de texto** (como el Bloc de notas, pero que funciona directo en la terminal negra). Cuando la guía dice que uses `nano`, te está pidiendo que abras ese editor para redactar un archivo de texto nuevo.

**¿Qué es crear una "unidad del servicio" y cuál es su lógica?**

Piensa en tu servidor como una empresa. El archivo `datos.json` que guardaste en el paso anterior es el producto que vas a entregar. Sin embargo, para que alguien pueda consultar ese archivo por la red,

necesitas un trabajador que abra la ventanilla, espere las consultas y entregue los datos 24 horas al día, sin que tú tengas que estar sentado frente a la computadora ejecutando comandos manualmente.
Una **unidad de servicio** (el archivo que termina en `.service`) es literalmente el **manual de instrucciones o contrato** que tú redactas para el gerente general del sistema operativo (un programa interno llamado `systemd`). En ese archivo le explicas a Linux exactamente cómo debe crear a este trabajador invisible:

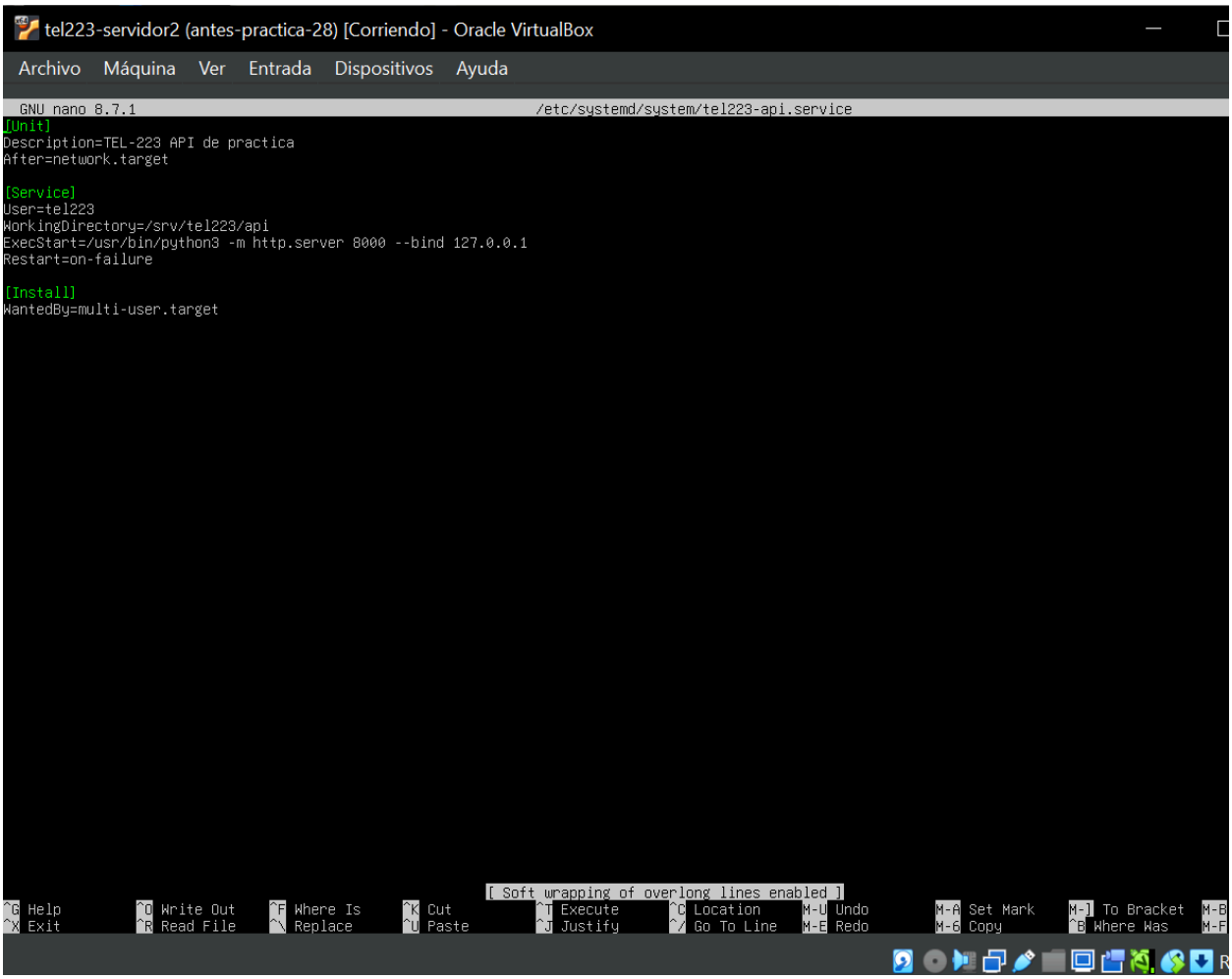
- Le dices qué usuario debe usar (`User=tel223`).
- Le dices en qué carpeta debe pararse a trabajar (`WorkingDirectory=/srv/tel223/api`).
- Le das el comando exacto para encender la API (`ExecStart=...`).
- Le dices qué hacer si el programa falla o se cae (`Restart=on-failure`).

Esto le da sentido al resto del laboratorio: al crear este archivo, tu API se ejecutará en segundo plano de manera profesional y automática. Estará siempre encendida, lista para que Nginx (tu puerta principal en la Etapa 4) le pase las peticiones de los clientes.
**¿Por qué este archivo se guarda en `/etc/` y no en `/srv/`?**
Linux es extremadamente organizado y separa estrictamente los datos de las configuraciones:

- `/srv/` **es tu bodega de productos:** Ahí es donde guardas los datos vivos de tu aplicación web (el `datos.json`).
- `/etc/` **es la oficina de administración central:** En Linux, la carpeta `/etc/` es el archivero oficial donde se guardan **todas las configuraciones del sistema operativo**.

El gerente del sistema (`systemd`) no va a buscar manuales de instrucciones a la bodega de productos (`/srv/`). Está programado desde fábrica para buscar los archivos `.service` única y exclusivamente en su archivero administrativo, que es la ruta

`/etc/systemd/system/`. Si guardaras este archivo en `/srv/`, Linux jamás lo encontraría y tu servicio no podría arrancar.




(el contenido está cortado porque se sacó la captura de pantalla después de otros comandos, pero, está correctamente implementado)

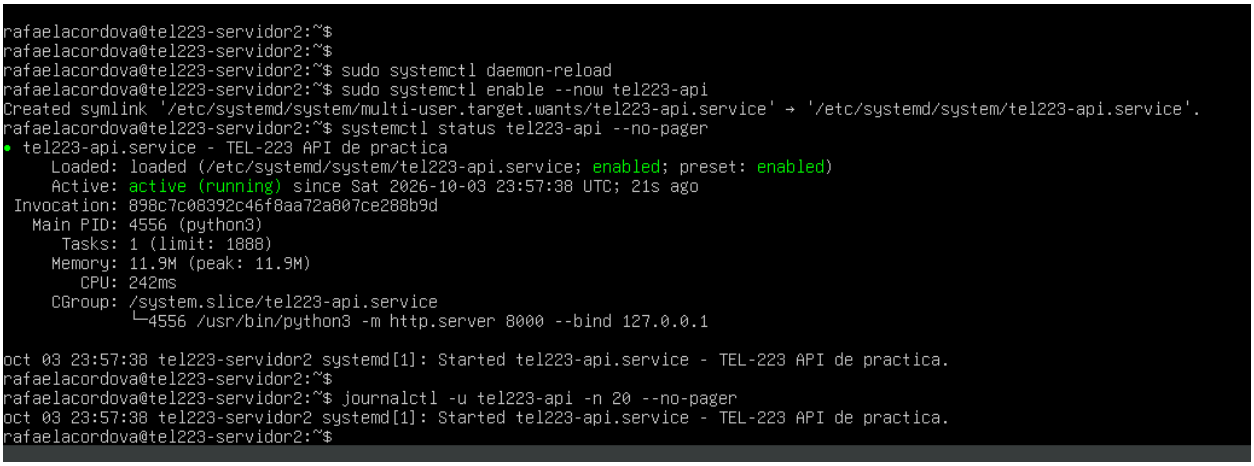
**7.** Cargue la unidad, habilítela, iníciela y compruebe su estado y su registro:

```
sudo systemctl daemon-reload
sudo systemctl enable --now tel223-api
systemctl status tel223-api --no-pager
journalctl -u tel223-api -n 20 --no-pager
```

▼ Comandos systemctl status ... && journalctl -u...

```
rafaelacordova@tel223-servidor2:~$
rafaelacordova@tel223-servidor2:~$
rafaelacordova@tel223-servidor2:~$ sudo systemctl daemon-reload
rafaelacordova@tel223-servidor2:~$ sudo systemctl enable --now tel223-api
Created symlink '/etc/systemd/system/multi-user.target.wants/tel223-api.service' → '/etc/systemd/system/tel223-api.service'.
rafaelacordova@tel223-servidor2:~$ systemctl status tel223-api --no-pager
● tel223-api.service - TEL-223 API de practica
     Loaded: loaded (/etc/systemd/system/tel223-api.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-03 23:57:38 UTC; 21s ago
 Invocation: 898c7c08392c46f8aa72a807ce288b9d
   Main PID: 4556 (python3)
      Tasks: 1 (limit: 1888)
     Memory: 11.9M (peak: 11.9M)
        CPU: 242ms
     CGroup: /system.slice/tel223-api.service
             └─4556 /usr/bin/python3 -m http.server 8000 --bind 127.0.0.1

oct 03 23:57:38 tel223-servidor2 systemd[1]: Started tel223-api.service - TEL-223 API de practica.
rafaelacordova@tel223-servidor2:~$
rafaelacordova@tel223-servidor2:~$ journalctl -u tel223-api -n 20 --no-pager
oct 03 23:57:38 tel223-servidor2 systemd[1]: Started tel223-api.service - TEL-223 API de practica.
rafaelacordova@tel223-servidor2:~$
```




▼ Diagnóstico de la salida

💡 El resultado de la imagen es **absolutamente correcto y exitoso**. Lograste configurar y encender el servicio a la perfección, cumpliendo exactamente con lo que pide la Etapa 1 de la guía. Aquí tienes la explicación lógica de lo que el sistema te acaba de responder en la pantalla:

- **El mensaje `Created symlink...`:** Al ejecutar `enable --now`, el sistema creó un acceso directo (un enlace simbólico) en su carpeta de arranque automátio. La lógica de esto es que, si tu máquina virtual se apaga o se reinicia, ya no tendrás que volver a

encender la API a mano; el sistema operativo lo hará por ti de forma automática.

- **El estado `active (running)` en verde:** Esta es la señal vital más importante que arroja el comando `status`. Te confirma que tu "trabajador" (el programa de Python que lee el archivo JSON) no solo encendió, sino que está vivo, trabajando en segundo plano, y consumiendo memoria RAM (11.9M) sin haberse estrellado.
- **La salida de `journalctl`:** Al revisar el diario de la caja negra, la última línea que te muestra el sistema es simplemente `Started tel223-api.service - TEL-223 API de practica.`. La lógica de ver solo este mensaje es excelente: significa que el arranque fue totalmente limpio y no se registró ningún error, permiso denegado o falla en el camino.

Tu servicio ya está corriendo y listo para el siguiente paso, que será comprobar por qué puerto de red está escuchando.

**8.** Compruebe que escucha y que responde, desde el propio servidor:

```
sudo ss -ltnp | grep 8000
```

▼ Comando sudo ss...



El comando `sudo ss -ltnp | grep 8000` actúa como un radar de red para confirmar si tu servicio realmente logró abrir su puerta de comunicaciones y está esperando conexiones.
Aquí tienes el desglose lógico de cada componente:

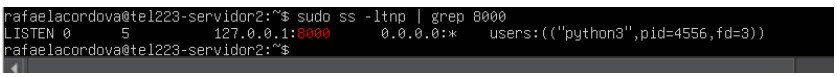
- `ss` **(Socket Statistics):** Es la herramienta principal y moderna en Linux para inspeccionar el estado de las conexiones de red (reemplaza al antiguo comando `netstat`).
- **Los modificadores `ltnp`:**
  - `l` **(Listening / Escuchando):** Filtra los resultados para mostrar únicamente los puertos que están "a la escucha"; es

decir, aquellos que están abiertos, inactivos y esperando pacientemente que un cliente se conecte.

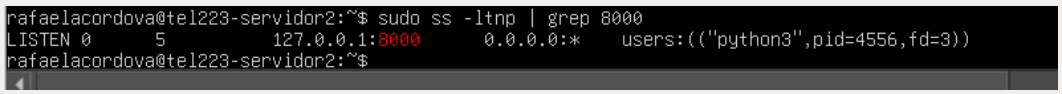
    - `t` **(TCP):** Le indica que solo busque puertos que usen el protocolo de transporte TCP, que es el estándar confiable que utilizan las páginas web y las APIs.
    - `n` **(Numeric / Numérico):** Obliga al sistema a mostrar las direcciones IP y los números de los puertos de forma cruda (ej. `8000`), bloqueando cualquier intento del sistema de traducir esos números a nombres amigables. Esto hace que la consulta sea exacta y sin demoras.
    - `p` **(Processes / Procesos):** Muestra el nombre y el número de identificación (PID) del programa exacto que se adueñó de ese puerto. *(Nota: es precisamente por este modificador que necesitas usar `sudo`, ya que los usuarios normales no tienen permiso para espiar qué procesos corren detrás de los puertos del sistema)*.

- `|` **(Tubería o Pipe):** Toma toda la lista extensa de puertos abiertos que arrojó `ss` y se la inyecta como texto al siguiente comando en lugar de imprimirla directamente en tu pantalla.
- `grep 8000`**:** Es el buscador y filtrador de texto por excelencia en Linux. Examina la lista que recibió por la tubería y te devuelve exclusivamente la línea que contiene el número `8000`.

**Su sentido de existencia en el laboratorio:**
Aunque en el paso anterior el comando `status` te confirmó que el programa estaba `active (running)`, un programa puede estar encendido pero desconectado de la red (por ejemplo, si el puerto 8000 ya estaba ocupado por otro servicio). Este comando es la prueba física e irrefutable a nivel de red de que tu API se ancló correctamente al puerto `8000` y está lista para recibir tráfico.



▼ Análisis de la salida




```
curl http://127.0.0.1:8000/datos.json
```

**Qué debe ver:** el servicio active (running), un socket en 127.0.0.1:8000 y el contenido JSON. Escucha solo en 127.0.0.1 a propósito: no debe ser accesible desde fuera; más adelante Nginx le reenviará las solicitudes.




| Pregunta de repaso | Mi respuesta |
|---|---|
| ¿Por qué el servicio corre con un usuario propio y sin shell, y no como root? | Se le asigna un usuario propio para aislar el servicio de todo el control del servidor, a un directorio restringido. Sin shell, cualquier individuo que ingrese al usuario tendría el control sobre todo el servidor (root) a través de la línea de comandos. |
| ¿Qué diferencia hay entre enable y start? | start: solo inicia el servidor en la ejecución actual de la VM. Si se apaga, el servidor volverá a apagarse.<br>enable: activa el servidor para que arranque solo cada vez que la VM se encienda. |
| **Control de la etapa —** tel223-api activo, escuchando en 127.0.0.1:8000 y respondiendo con el JSON. | |

# ETAPA 2

**ETAPA 2 — Direccionamiento y conectividad**

*Objetivo: confirmar que el servidor tiene su dirección fija en la red interna y que el cliente lo alcanza. Si lo dejó listo en la práctica anterior, esta etapa es solo verificación.*

**9.** En el SERVIDOR, revise la configuración de red existente antes de tocar nada. Netplan combina todos los archivos YAML del directorio: no borre archivos sin revisarlos.

(REVISIÓN DE CONFIGURACIONES DE ANTERIORES LABORATORIOS)

```
ls -l /etc/netplan
sudo netplan get
ip -br addr
```

▼ Lógica de los comandos

Estos tres comandos conforman una rutina de inspección fundamental en redes. Su objetivo en el paso 9 es permitirte observar el estado actual del servidor antes de realizar cualquier cambio, evitando conflictos con configuraciones antiguas.

Aquí tienes el desglose lógico de cada comando y sus componentes:

### 1. `ls -l /etc/netplan`

Este comando sirve para **inspeccionar los archivos físicos** de configuración de red que existen en el disco duro.

- `ls` **(Listar):** Es el comando básico para ver el contenido de una carpeta.
- `l` **(Formato largo):** Modificador que te muestra los resultados en una lista detallada (incluyendo permisos, propietario y fecha de modificación) en lugar de solo los nombres de los archivos.
- `/etc/netplan`**:** Es la ruta oficial donde Ubuntu guarda los archivos `.yaml` que dictan cómo deben comportarse las tarjetas de red.
- **Su sentido lógico:** Como indica la guía, la herramienta Netplan combina absolutamente todos los archivos que encuentre en esta carpeta. Si tienes un archivo viejo de una práctica anterior y creas uno nuevo, Netplan mezclará ambos y la red fallará. Este comando te permite ver exactamente qué archivos hay para que sepas qué editar o eliminar.

### 2. `sudo netplan get`

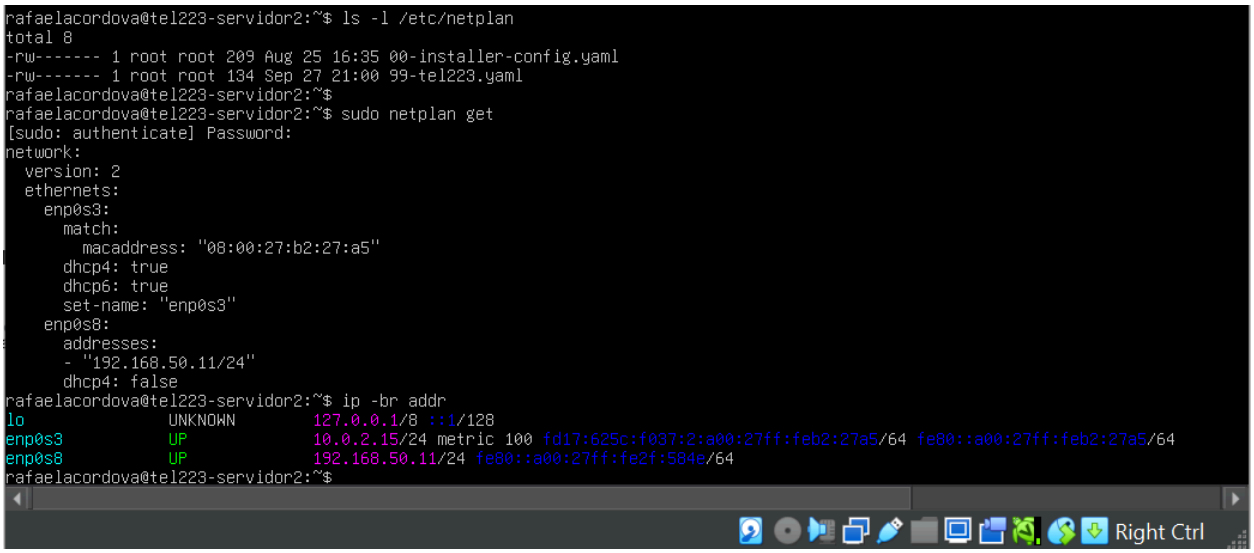
Este comando sirve para **ver el resumen lógico** de la red tal como lo entiende el sistema operativo.

- `sudo`**:** Invoca permisos de administrador, necesarios para leer configuraciones críticas del sistema.
- `netplan`**:** Es el programa gestor de red predeterminado en Ubuntu.
- `get` **(Obtener):** Le pide a Netplan que lea todos los archivos `.yaml` de la carpeta, los procese, los unifique y te imprima en pantalla el resultado final de la configuración.
- **Su sentido lógico:** En lugar de abrir archivo por archivo para leerlos con `nano`, este comando te muestra de un solo vistazo el mapa consolidado de cómo el servidor *cree* que debe configurar la red.

### 3. `ip -br addr`

Este comando sirve para **ver la realidad física** de tus tarjetas de red en este preciso instante.

- `ip`: Es la herramienta principal de Linux para gestionar la red.
- `br` **(Brief / Breve):** Obliga a la terminal a mostrarte un resumen limpio, dedicando una sola línea por cada tarjeta de red.
- `addr` **(Address / Direcciones):** Filtra la información para mostrarte únicamente las direcciones IP.
- **Su sentido lógico:** Mientras que `netplan get` te dice qué es lo que el sistema *intenta* hacer, `ip -br addr` te muestra lo que *realmente* está sucediendo. Te confirma si la tarjeta lógicamente encendió (`UP`) y si efectivamente logró asignarse la dirección IP esperada (como la `.11`).




▼ Análisis de los resultados

El primer comando, `ls -l /etc/netplan`, es como abrir el cajón oficial donde el servidor guarda sus manuales de instrucciones de red. El

resultado en pantalla te muestra que existen exactamente dos documentos guardados allí: uno original que trajo el sistema al instalarse (`00-installer-config.yaml`) y el que tú configuraste para esta práctica (`99-tel223.yaml`).
El segundo comando, `sudo netplan get`, es la interpretación que hace el sistema. Te muestra la conclusión final a la que llega el servidor tras leer y combinar esos dos manuales:

- Para la primera tarjeta de red (`enp0s3`), el sistema entendió la orden `dhcp4: true`. Esto significa: *"Consigue tu dirección de forma automática para poder salir a internet"*.
- Para la segunda tarjeta (`enp0s8`), el sistema entendió que tiene asignada una dirección de forma permanente y estricta (`192.168.50.11`) y que tiene prohibido pedir direcciones automáticas (`dhcp4: false`). Esta es tu red interna y privada del laboratorio.

El tercer comando, `ip -br addr`, es la revisión de la realidad. Te confirma visualmente que las instrucciones teóricas de arriba se aplicaron con éxito en el mundo físico:

- Ambas tarjetas de red te dicen `UP` en color verde, lo que significa que están encendidas, operativas y con el "cable" virtual conectado.
- La tarjeta `enp0s3` logró conseguir automáticamente su dirección para internet (`10.0.2.15`).
- La tarjeta `enp0s8` tiene anclada exactamente la dirección `192.168.50.11` que esperabas.

En resumen, los resultados te indican que todo el sistema de red de tu servidor está configurado a la perfección: los manuales existen, el servidor los entendió bien, y las tarjetas físicas los están obedeciendo al pie de la letra.

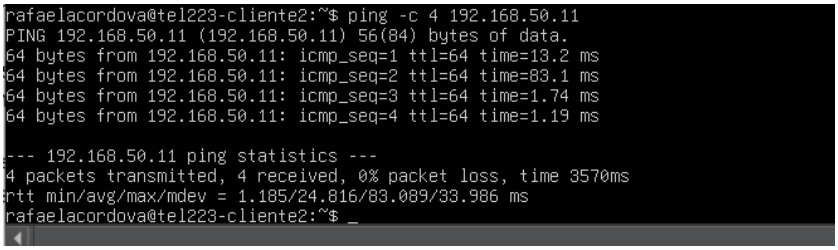
ip route

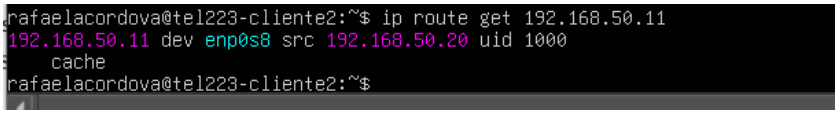
**11.** En el CLIENTE, por ahora con su dirección fija 192.168.50.20/24 de la práctica anterior, compruebe el alcance y la ruta hacia el servidor:

```
ping -c 4 192.168.50.11
ip route get 192.168.50.11
```

**Qué debe ver:** 0 % de pérdida y una ruta que sale por el adaptador interno. La ruta por defecto de las dos máquinas sigue saliendo por NAT.

**Control de la etapa —** servidor en 192.168.50.11, ping del cliente con 0 % de pérdida y NAT intacto.



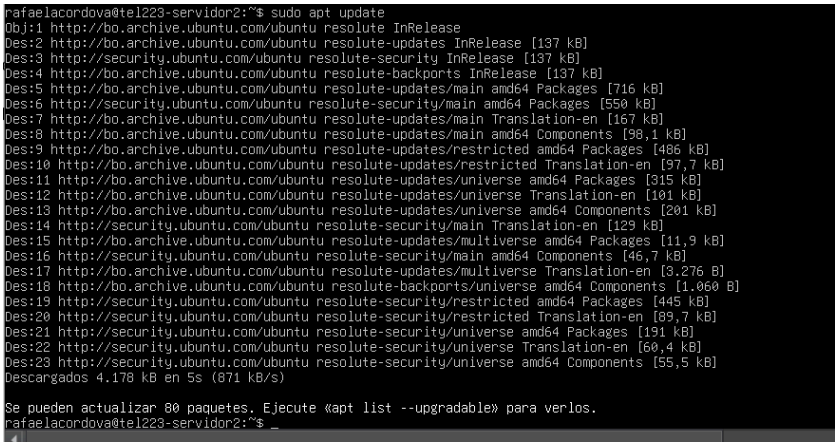

# ETAPA 3

### ETAPA 3 — DNS y DHCP con dnsmasq

*Objetivo: que el servidor resuelva los nombres de la zona tel223.test y entregue direcciones al cliente. Un solo servicio cumple las dos funciones.*

**12.** En el SERVIDOR, instale dnsmasq y las utilidades de consulta. Es normal que al instalarse muestre un error de arranque: todavía no tiene la configuración de hoy.

```
sudo apt update
```



```
rafaelacordova@tel223-servidor2:~$ sudo apt install -y dnsmasq dnsutils
Nota, seleccionando «bind9-dnsutils» en lugar de «dnsutils»
bind9-dnsutils ya está en su versión más reciente (1:9.20.24-1ubuntu0.3).
fijado bind9-dnsutils como instalado manualmente.
Instalando:
  dnsmasq

Instalando dependencias:
  dns-root-data  dnsmasq-base

Resumen:
  Actualizando: 0, Instalando 3, Eliminando: 0, no actualizando: 80
  Tamaño de la descarga: 463 kB
  Espacio necesario: 1.051 kB / 4.622 MB disponible

Des:1 http://bo.archive.ubuntu.com/ubuntu resolute/main amd64 dns-root-data all 2025080400build1 [6.022 B]
Des:2 http://bo.archive.ubuntu.com/ubuntu resolute-updates/main amd64 dnsmasq-base amd64 2.92-1ubuntu0.4 [439 kB]
Des:3 http://bo.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 dnsmasq all 2.92-1ubuntu0.4 [17,9 kB]
Descargados 463 kB en 1s (524 kB/s)
Seleccionando el paquete dns-root-data previamente no seleccionado.
(Leyendo la base de datos ... 139624 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../dns-root-data_2025080400build1_all.deb ...
Desempaquetando dns-root-data (2025080400build1) ...
Seleccionando el paquete dnsmasq-base previamente no seleccionado.
Preparando para desempaquetar .../dnsmasq-base_2.92-1ubuntu0.4_amd64.deb ...
Desempaquetando dnsmasq-base (2.92-1ubuntu0.4) ...
Seleccionando el paquete dnsmasq previamente no seleccionado.
Preparando para desempaquetar .../dnsmasq_2.92-1ubuntu0.4_all.deb ...
Desempaquetando dnsmasq (2.92-1ubuntu0.4) ...
Configurando dnsmasq-base (2.92-1ubuntu0.4) ...
Configurando dns-root-data (2025080400build1) ...
Configurando dnsmasq (2.92-1ubuntu0.4) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/dnsmasq.service' → '/usr/lib/systemd/system/dnsmasq.service'.
Could not execute systemctl:  at /usr/bin/deb-systemd-invoke line 148.
Procesando disparadores para man-db (2.13.1-1build1) ...
Procesando disparadores para dbus (1.16.2-2ubuntu4) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
rafaelacordova@tel223-servidor2:~$ _
```

13. Revise si quedaron archivos de prácticas anteriores. dnsmasq lee todos los archivos de esa carpeta y los combina:

```
ls /etc/dnsmasq.d
```

**Si aparece otro archivo:** además de README, muévalo a su carpeta personal para que no interfiera: sudo mv /etc/dnsmasq.d/NOMBRE.conf ~/

```
rafaelacordova@tel223-servidor2:~$ ls /etc/dnsmasq.d
README
rafaelacordova@tel223-servidor2:~$
```

14. Cree la configuración. Si tel223.conf ya existe de una práctica anterior, reemplace todo su contenido por este:

```
sudo nano /etc/dnsmasq.d/tel223.conf
```

```
interface=enp0s8
bind-interfaces
domain=tel223.test
address=/portal.tel223.test/192.168.50.11
dhcp-range=192.168.50.50,192.168.50.60,12h
dhcp-option=option:dns-server,192.168.50.11
dhcp-option=3
```

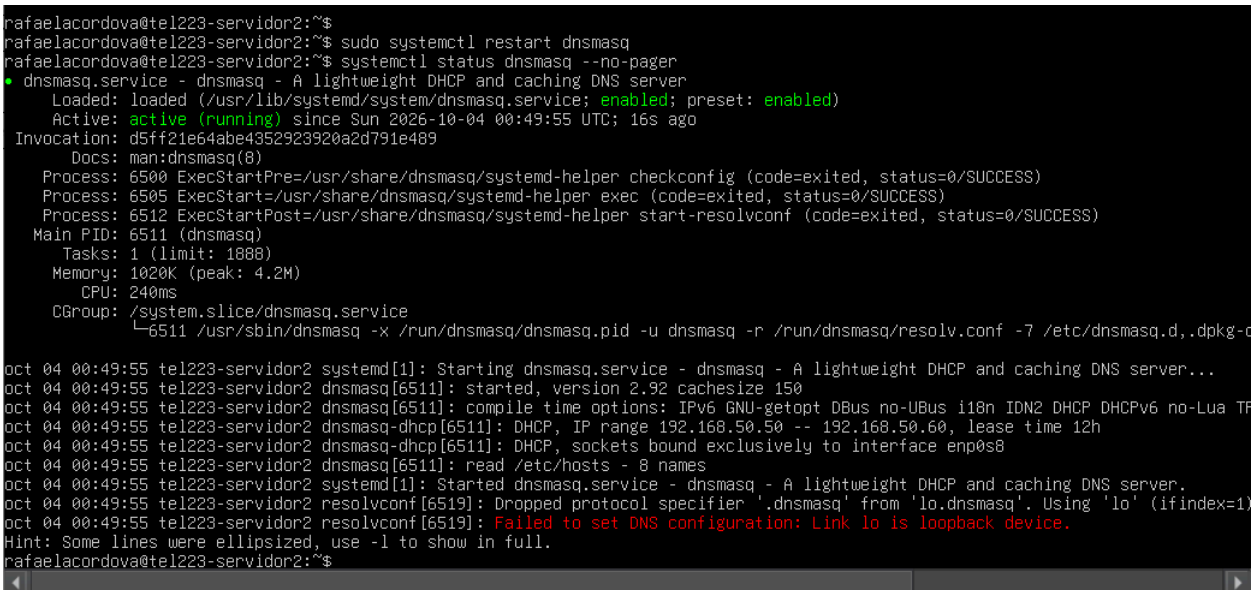
```
rafaelacordova@tel223-servidor2:~$ sudo nano /etc/dnsmasq.d/tel223.conf
```

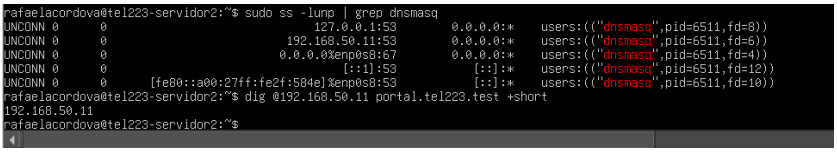
```
tel223-servidor2 (antes-practica-28) [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
  GNU nano 8.7.1                                                /etc/
interface=enp0s8
bind-interfaces
domain=tel223.test
address=/portal.tel223.test/192.168.50.11
dhcp-range=192.168.50.50,192.168.50.60,12h
dhcp-option=option:dns-server,192.168.50.11
dhcp-option=3_
```

| Línea | Qué hace |
|---|---|
| interface / bind-interfaces | Escucha solo en la tarjeta interna, nunca en la de NAT |
| address | Registro del nombre portal.tel223.test hacia la dirección del servidor |
| dhcp-range | Rango dinámico .50 a .60. La .11 del servidor queda fuera del rango |
| dhcp-option=option:dns-server | Indica al cliente qué servidor DNS usar |
| dhcp-option=3 | No anuncia puerta de enlace: la salida a Internet del cliente sigue por NAT |

**15.** Reinicie, verifique y consulte al DNS interno desde el propio servidor:

```
sudo systemctl restart dnsmasq
systemctl status dnsmasq --no-pager
sudo ss -lunp | grep dnsmasq
dig @192.168.50.11 portal.tel223.test +short
```




▼ Análisis de respuestas



Sí, tus resultados son **absolutamente correctos y un éxito total**. Has logrado configurar tu servidor para que actúe como repartidor de direcciones (DHCP) y como traductor de nombres (DNS) al mismo tiempo.

Aquí tienes el análisis lógico de por qué tus salidas en la pantalla demuestran que todo está perfecto:

**1. La salida de** `systemctl status`

- `active (running)`: Te confirma que el servicio `dnsmasq` encendió sin errores y está trabajando en segundo plano.
- **La lectura del diario (Logs):** Si observas las últimas líneas del reporte, el propio sistema te confiesa lo que hizo al arrancar:
  - Dice `DHCP, IP range 192.168.50.50 -- 192.168.50.60`. Esto significa que leyó perfectamente tu configuración y preparó ese "paquete" de 11 direcciones IP para prestarlas a los clientes.
  - Dice `read /etc/hosts - 8 names`. Confirma que `dnsmasq` fue a leer el archivo interno de nombres de tu servidor y aprendió 8 traducciones de memoria.

**2. La salida de `ss -lunp | grep dnsmasq`**

- A diferencia de tu API que usaba TCP (y por eso usabas el modificador `t`), DNS y DHCP usan el protocolo más rápido **UDP**. Por eso el comando ahora tiene la letra `u` (`lunp`).
- `192.168.50.11:53`: El puerto `53` es la ventanilla universal de Internet para el servicio DNS. Esto te confirma que tu servidor está escuchando en su IP interna y listo para traducir nombres.
- `0.0.0.0%enp0s8:67`: El puerto `67` es la ventanilla universal para el servicio DHCP. Aquí te confirma que el repartidor de IPs está anclado específicamente a la tarjeta de red interna (`enp0s8`) listo para atender a los clientes de tu laboratorio.

**3. La salida del comando `dig`**

- **La lógica del comando:** `dig` es la herramienta forense para hacerle preguntas directas a un servidor DNS. Al usar `@192.168.50.11` le dijiste *"No le preguntes a internet, pregúntale exactamente a mi servidor interno"* y le pediste que tradujera la dirección `portal.tel223.test`. El modificador `+short` sirve para que no te arroje toda la burocracia del protocolo y te dé solo la respuesta final.

- **El resultado:** La terminal te devolvió de forma limpia `192.168.50.11`. Esta es la prueba de oro. Tu DNS interno acaba de funcionar: tomó un nombre de dominio inventado por ti y lo tradujo exitosamente a la IP de tu servidor.

Tu infraestructura base ya está completamente lista.

▼ Explicación de la Falla (último registro) en status



¡Qué buen ojo! Es un excelente hábito de administrador de sistemas leer detenidamente los registros y no pasar por alto las letras rojas. La línea exacta a la que te refieres dice: `resolvconf[6519]: Failed to set DNS configuration: Link lo is loopback device.`

No tienes de qué preocuparte, **este error es inofensivo** y no afecta en nada a tu laboratorio. Aquí tienes la lógica de por qué ocurre y por qué lo ignoramos:

**¿Qué intentó hacer dnsmasq?**

Por defecto, cuando `dnsmasq` se enciende, no solo se prepara para atender a las computadoras cliente, sino que también intenta ser "demasiado servicial" y trata de configurarse a sí mismo como el traductor DNS principal *para el propio servidor*. Para lograrlo, intenta inyectar su configuración en la tarjeta de red interna virtual del servidor (llamada `lo` o loopback).

**¿Por qué falló?**

Falló porque los sistemas Linux modernos (como Ubuntu) ya tienen un gerente interno muy estricto que controla la red (llamado `systemd-resolved`). Cuando `dnsmasq` intentó modificar la configuración DNS de la tarjeta `lo`, este gerente interno lo bloqueó y le dijo: *"Operación fallida, no puedes modificar la configuración DNS de `lo` porque es una tarjeta loopback reservada para el sistema"*.

**¿Por qué no importa para el laboratorio?**

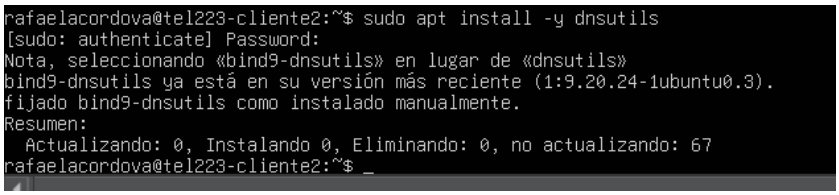
Ese pequeño bloqueo solo le impidió a `dnsmasq` modificarse a sí mismo de manera interna. Sin embargo, su misión real en esta

práctica no es traducirse cosas a sí mismo, sino **traducir nombres y entregar direcciones IP a las computadoras de la red** `tel223-lab`. Como comprobaste con los comandos `ss` y `dig` de la imagen, `dnsmasq` logró anclarse exitosamente a la tarjeta de red correcta (`enp0s8`) y está respondiendo perfectamente hacia el exterior. Por lo tanto, puedes ignorar esa advertencia de `resolvconf` y continuar con total seguridad a configurar tu cliente.

**16.** En el CLIENTE, instale las utilidades y cambie el adaptador interno a DHCP editando su /etc/netplan/99-tel223.yaml:

```
sudo apt install -y dnsutils
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: true
sudo netplan generate
sudo netplan try
ip -br addr
ip route
```

**Qué debe ver:** una dirección entre 192.168.50.50 y 192.168.50.60 en el adaptador interno, y la ruta por defecto todavía por el adaptador de NAT.




▼ Análisis de la respuesta

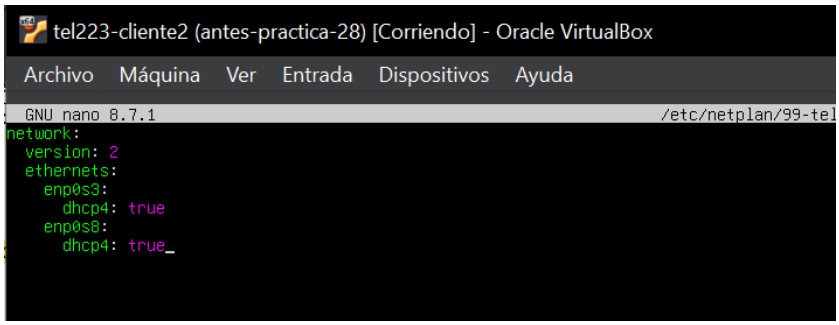
💡 Sí, la respuesta en tu terminal es **totalmente correcta y normal**. No hay ningún error.
El sistema simplemente te está avisando que el conjunto de herramientas que intentaste instalar **ya existía** previamente en tu computadora cliente. Aquí tienes la lógica de lo que el sistema te respondió:

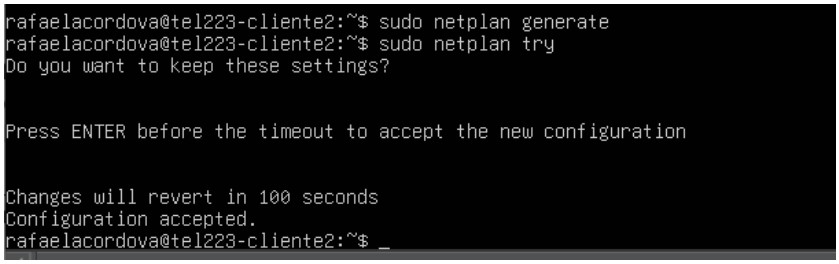
- `seleccionando «bind9-dnsutils» en lugar de...`: El sistema operativo Ubuntu actualizó el nombre oficial de este paquete en sus repositorios, así que el instalador hizo la corrección por ti de forma automática.

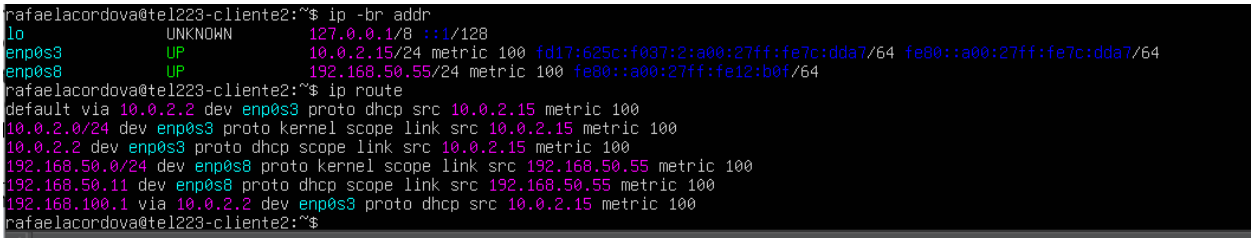
- `ya está en su versión más reciente`: Te confirma que los programas de red que vienen en ese paquete (específicamente el comando `dig` que necesitarás más adelante para probar el DNS) ya están instalados y listos para usarse en este cliente.
- `Actualizando: 0, Instalando 0`: Como el programa ya estaba ahí, el gestor de paquetes no tuvo necesidad de descargar ni instalar absolutamente nada nuevo en tu disco duro.

Puedes continuar con total seguridad al siguiente paso de tu guía.



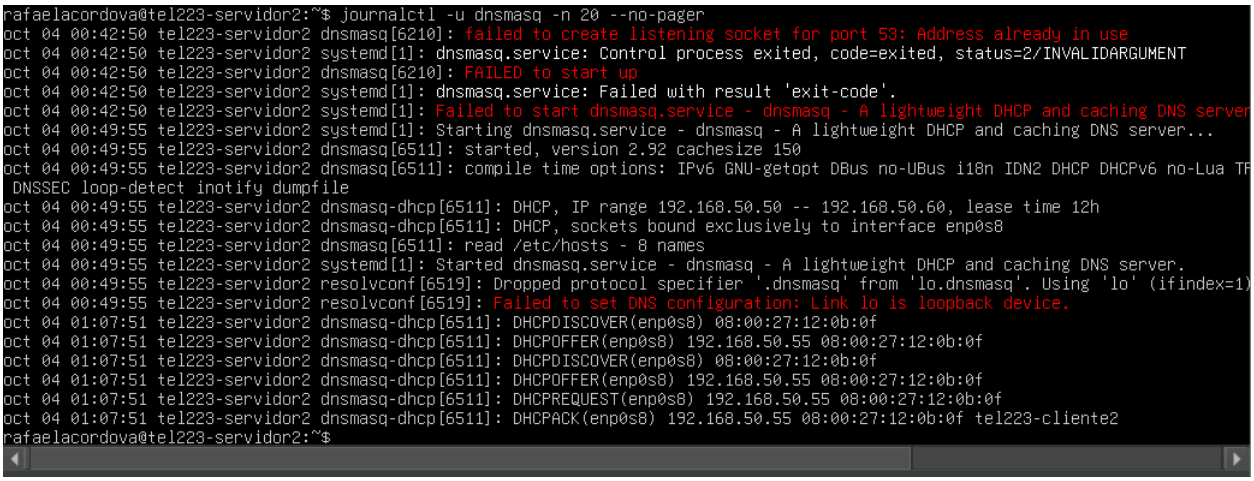




**17.** En el SERVIDOR, lea en el registro el intercambio DHCP que acaba de ocurrir:

```
journalctl -u dnsmasq -n 20 --no-pager
```




▼ Errores del puerto 53



Esos mensajes rojos son el clásico error informático de "dos personas intentando sentarse en la misma silla".
El error principal dice literalmente: `failed to create listening socket for port 53: Address already in use`.
Aquí tienes la lógica de lo que pasó en ese momento:

- **El choque de puertos:** Como vimos antes, el puerto `53` es la ventanilla universal para el servicio DNS. Cuando tu servicio `dnsmasq` intentó encender, descubrió que otro programa ya estaba adueñado de ese puerto (en los sistemas Ubuntu, esto casi siempre ocurre porque el sistema operativo trae su propio gestor de DNS activado por defecto, o porque un intento anterior de `dnsmasq` se quedó atascado).
- **El colapso:** Al encontrar su ventanilla de trabajo ocupada y no poder escuchar peticiones, el programa tiró la toalla y se apagó. Eso es lo que te informan las siguientes líneas rojas de `FAILED to start up` (Fallo al arrancar).

**¿Por qué no te afectó en el laboratorio?**
La clave está en las horas (timestamps) que aparecen al inicio de cada línea. Ese colapso ocurrió a las `00:42:50`. Sin embargo, siete

minutos después, a las `00:49:55`, el registro te muestra que el servicio arrancó de forma limpia y brillante.
Esto significa que, entre un evento y otro, aplicaste la configuración correcta (probablemente la instrucción de la guía que limita a `dnsmasq` para que solo escuche en la tarjeta `enp0s8`), liberaste el conflicto, y el servidor quedó operando a la perfección.
¿Te parece si ahora pasamos a la prueba final y comprobamos con el comando `dig` si el cliente ya es capaz de aprovechar las traducciones de este DNS?



Esos mensajes rojos son el clásico error informático de "dos personas intentando sentarse en la misma silla".
El error principal dice literalmente: `failed to create listening socket for port 53: Address already in use`.
Aquí tienes la lógica de lo que pasó en ese momento:

- **El choque de puertos:** Como vimos antes, el puerto `53` es la ventanilla universal para el servicio DNS. Cuando tu servicio `dnsmasq` intentó encender, descubrió que otro programa ya estaba adueñado de ese puerto (en los sistemas Ubuntu, esto casi siempre ocurre porque el sistema operativo trae su propio gestor de DNS activado por defecto, o porque un intento anterior de `dnsmasq` se quedó atascado).
- **El colapso:** Al encontrar su ventanilla de trabajo ocupada y no poder escuchar peticiones, el programa tiró la toalla y se apagó. Eso es lo que te informan las siguientes líneas rojas de `FAILED to start up` (Fallo al arrancar).

**¿Por qué no te afectó en el laboratorio?**
La clave está en las horas (timestamps) que aparecen al inicio de cada línea. Ese colapso ocurrió a las `00:42:50`. Sin embargo, siete minutos después, a las `00:49:55`, el registro te muestra que el servicio arrancó de forma limpia y brillante.

Esto significa que, entre un evento y otro, aplicaste la configuración correcta (probablemente la instrucción de la guía que limita a `dnsmasq` para que solo escuche en la tarjeta `enp0s8`), liberaste el conflicto, y el servidor quedó operando a la perfección.
¿Te parece si ahora pasamos a la prueba final y comprobamos con el comando `dig` si el cliente ya es capaz de aprovechar las traducciones de este DNS?

▼ Errores efecto



Esos dos mensajes en rojo son simplemente el "efecto dominó" o la consecuencia directa del primer error del puerto ocupado. No son problemas nuevos, sino el sistema reportando el colapso en diferentes niveles organizativos.
Aquí tienes la lógica de cada línea:

1. `FAILED to start up`: Este es el último mensaje emitido por el propio programa `dnsmasq`. Al darse cuenta de que la ventanilla 53 estaba ocupada y no podría hacer su trabajo, su código interno determina que es inútil seguir operando, por lo que emite esta alerta de fallo fatal y detiene sus propios procesos de inmediato.
2. `Failed to start dnsmasq.service...`: Este mensaje ya no proviene del programa, sino de `systemd` (el gerente general del sistema operativo que observamos en la etapa 1). Como `systemd` fue quien intentó arrancar la unidad de servicio, al ver que el trabajador colapsó y devolvió un código de error, el gerente levanta un acta oficial en letras rojas declarando a nivel de sistema que el arranque de esa unidad de servicio fracasó por completo.

En resumen: el primer error es la **causa** (puerto ocupado), el segundo es la **reacción del programa** (rendirse y apagarse), y el tercero es el **acta oficial** que emite el sistema operativo confirmando el fallo del servicio.

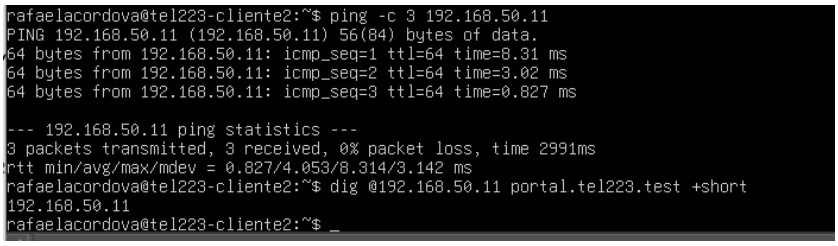
**18.** En el CLIENTE, compruebe alcance y resolución preguntando al DNS interno:

```
ping -c 3 192.168.50.11
dig @192.168.50.11 portal.tel223.test +short
```

**Por qué se indica @192.168.50.11:** los nombres .test no existen en la jerarquía pública; se resuelven solo en el DNS interno del laboratorio. Preguntarle directamente evita depender del resolvedor de la máquina.

**Reto opcional — reserva DHCP:** averigüe la MAC del adaptador interno del cliente (ip link show enp0s8) y agregue en el servidor la línea dhcp-host=MAC,192.168.50.20. Reinicie dnsmasq, renueve en el cliente con sudo netplan apply y compruebe que recibe la .20. Si conserva la dirección anterior, reinicie el cliente con sudo reboot.

**Control de la etapa —** el cliente recibe dirección por DHCP, el registro muestra los cuatro mensajes y portal.tel223.test resuelve a 192.168.50.11.

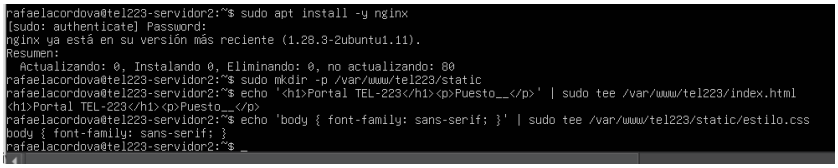



# ETAPA 4



**19.** En el SERVIDOR, instale Nginx (si ya lo tiene, no pasa nada) y cree el contenido:

```
sudo apt install -y nginx
sudo mkdir -p /var/www/tel223/static
echo '<h1>Portal TEL-223</h1><p>Puesto __</p>' | sudo tee /var/www/tel223/index.html
echo 'body { font-family: sans-serif; }' | sudo tee /var/www/tel223/static/estilo.css
```




**20.** Cree el sitio:

```
sudo nano /etc/nginx/sites-available/tel223
server {
    listen 80 default_server;
    server_name portal.tel223.test;
    root /var/www/tel223;
    index index.html;

    location /static/ {
        try_files $uri =404;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8000/;
    }
}
```

(se puede escribir con tabulaciones)

```
rafaelacordova@tel223-servidor2:~$ sudo nano /etc/nginx/sites-available/tel223_
```

tel223-servidor2 (antes-practica-28) [Corriendo] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

```
 GNU nano 8.7.1                                  /etc/nginx/sites-available/tel223 *
server {
        listen 80 default_server;
        server_name portal.tel223.test;
        root /var/www/tel223;
        index index.html;

        location /static/ {
                try_files $uri =404;
        }

        location /api/ {
                proxy_pass http://127.0.0.1:8000/;
        }
}
```

**21.** Active el sitio y desactive el de ejemplo. Lo que se borra es solo el enlace en sites-enabled; el archivo original queda en sites-available.

```
sudo rm /etc/nginx/sites-enabled/default
sudo ln -s /etc/nginx/sites-available/tel223 /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
systemctl status nginx --no-pager
sudo ss -ltnp
```

**Qué debe ver:** nginx -t con syntax is ok y test is successful; Nginx escuchando en el 80 de todas las interfaces y tel223-api solo en 127.0.0.1:8000.

```
rafaelacordova@tel223-servidor2:~$ sudo rm /etc/nginx/sites-enabled/default
rafaelacordova@tel223-servidor2:~$ sudo ln -s /etc/nginx/sites-available/tel223 /etc/nginx/sites-enabled/
rafaelacordova@tel223-servidor2:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
rafaelacordova@tel223-servidor2:~$ sudo systemctl reload nginx
rafaelacordova@tel223-servidor2:~$ systemctl status nginx --no-pager
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-03 22:17:22 UTC; 3h 14min ago
 Invocation: 170e0d157e55425ab4356bd295aa9382
       Docs: man:nginx(8)
    Process: 7802 ExecReload=/usr/sbin/nginx -g daemon on; master_process on; -s reload (code=exited, status=0/SUCCESS)
   Main PID: 1016 (nginx)
      Tasks: 3 (limit: 1888)
     Memory: 4.8M (peak: 7.1M)
        CPU: 330ms
     CGroup: /system.slice/nginx.service
             ├─1016 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─7809 "nginx: worker process"
             └─7810 "nginx: worker process"

oct 03 22:17:20 tel223-servidor2 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
oct 03 22:17:22 tel223-servidor2 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
oct 04 01:31:26 tel223-servidor2 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server...
oct 04 01:31:26 tel223-servidor2 nginx[7802]: 2026/10/04 01:31:26 [notice] 7802#7802: signal process started
oct 04 01:31:26 tel223-servidor2 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
rafaelacordova@tel223-servidor2:~$ sudo ss -lntp
State  Recv-Q Send-Q                    Local Address:Port  Peer Address:Port Process
LISTEN 0      32                        192.168.50.11:53         0.0.0.0:*     users:(("dnsmasq",pid=6511,fd=7))
LISTEN 0      32                            127.0.0.1:53         0.0.0.0:*     users:(("dnsmasq",pid=6511,fd=9))
LISTEN 0      4096                       127.0.0.54:53         0.0.0.0:*     users:(("systemd-resolve",pid=591,fd=19))
LISTEN 0      5                             127.0.0.1:8000       0.0.0.0:*     users:(("python3",pid=4556,fd=3))
LISTEN 0      4096                      127.0.0.53%lo:53         0.0.0.0:*     users:(("systemd-resolve",pid=591,fd=17))
LISTEN 0      511                             0.0.0.0:80         0.0.0.0:*     users:(("nginx",pid=7810,fd=5),("nginx",pid=7809,f
LISTEN 0      4096                            0.0.0.0:22         0.0.0.0:*     users:(("systemd",pid=1,fd=143))
LISTEN 0      32     [fe80::a00:27ff:fe2f:584e]%enp0s8:53            [::]:*     users:(("dnsmasq",pid=6511,fd=11))
LISTEN 0      32                               [::1]:53            [::]:*     users:(("dnsmasq",pid=6511,fd=13))
LISTEN 0      4096                              [::]:22            [::]:*     users:(("systemd",pid=1,fd=144))
rafaelacordova@tel223-servidor2:~$ _
```

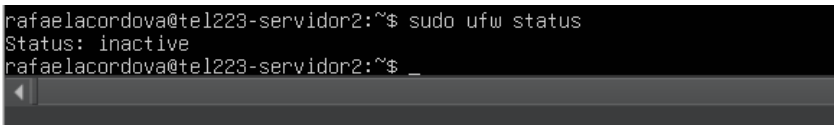
Right Ctrl

**22.** Revise el cortafuegos sin modificarlo:

```
sudo ufw status
```

**Si aparece active:** consulte al docente antes de cambiar nada. No lo desactive por su cuenta.

**Control de la etapa —** nginx -t sin errores, Nginx escuchando en el 80 y el sitio tel223 activo.




# ETAPA 5

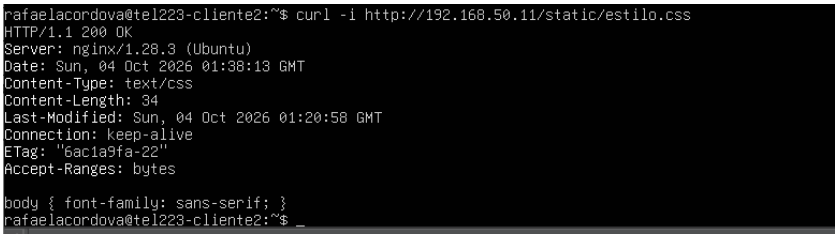
## ETAPA 5 — Lectura de transacciones HTTP y registros

*Objetivo: leer desde el CLIENTE las respuestas de cada ruta y relacionarlas con los registros del servidor.*

**23.** Ejecute cada consulta en el CLIENTE y complete la tabla:

```
curl -I http://192.168.50.11/
curl -i http://192.168.50.11/static/estilo.css
curl -i http://192.168.50.11/api/datos.json
curl -i http://192.168.50.11/noexiste
```






```
rafaelacordova@tel223-cliente2:~$ curl -i http://192.168.50.11/noexiste
HTTP/1.1 404 Not Found
Server: nginx/1.28.3 (Ubuntu)
Date: Sun, 04 Oct 2026 01:40:02 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive

<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>
rafaelacordova@tel223-cliente2:~$
```

**24.** Observe la conversación completa, petición y respuesta, con curl -v. Las líneas con > son la petición; las líneas con <, la respuesta:

```
curl -v http://192.168.50.11/api/datos.json
```

```
rafaelacordova@tel223-cliente2:~$ curl -v http://192.168.50.11/api/datos.json
*   Trying 192.168.50.11:80...
* Established connection to 192.168.50.11 (192.168.50.11 port 80) from 192.168.50.55 port 45046
* using HTTP/1.x
> GET /api/datos.json HTTP/1.1
> Host: 192.168.50.11
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.28.3 (Ubuntu)
< Date: Sun, 04 Oct 2026 01:41:39 GMT
< Content-Type: application/json
< Content-Length: 40
< Connection: keep-alive
< Last-Modified: Sat, 03 Oct 2026 23:17:51 GMT
<
{"servicio":"tel223-api","estado":"ok"}
* Connection #0 to host 192.168.50.11:80 left intact
rafaelacordova@tel223-cliente2:~$ _
```

```
rafaelacordova@tel223-servidor2:~$ sudo tail -n 6 /var/log/nginx/access.log
192.168.50.20 - - [27/Sep/2026:21:46:59 +0000] "GET / HTTP/1.1" 200 67 "-" "curl/8.18.0"
192.168.50.55 - - [04/Oct/2026:01:36:55 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/8.18.0"
192.168.50.55 - - [04/Oct/2026:01:38:13 +0000] "GET /static/estilo.css HTTP/1.1" 200 34 "-" "curl/8.18.0"
192.168.50.55 - - [04/Oct/2026:01:39:29 +0000] "GET /api/datos.json HTTP/1.1" 200 40 "-" "curl/8.18.0"
192.168.50.55 - - [04/Oct/2026:01:40:02 +0000] "GET /noexiste HTTP/1.1" 404 162 "-" "curl/8.18.0"
192.168.50.55 - - [04/Oct/2026:01:41:39 +0000] "GET /api/datos.json HTTP/1.1" 200 40 "-" "curl/8.18.0"
rafaelacordova@tel223-servidor2:~$ _
```

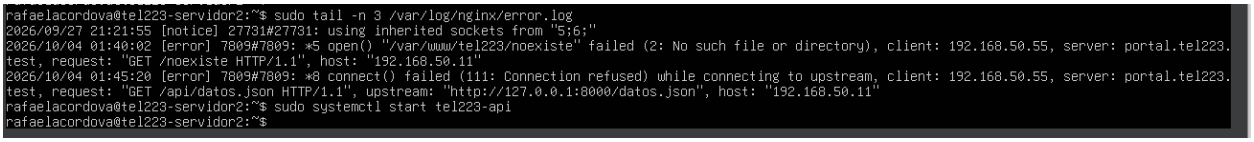
**26.** Un 5xx también es una respuesta. En el SERVIDOR detenga el backend; en el CLIENTE repita la consulta a /api/; luego lea el registro de errores y restaure:

```
sudo systemctl stop tel223-api            # servidor
curl -i http://192.168.50.11/api/datos.json   # cliente
sudo tail -n 3 /var/log/nginx/error.log   # servidor
sudo systemctl start tel223-api           # servidor
```

| Pregunta de repaso | Mi respuesta |
|---|---|
| ¿Qué código recibió con el backend detenido? ¿Estaba caído Nginx? | |
| ¿Qué dice error.log y qué capa señala? | |
| **Control de la etapa —** tabla completa, conversación leída con curl -v, líneas identificadas en access.log y el 502 explicado con error.log. | |

```
rafaelacordova@tel223-servidor2:~$ sudo systemctl stop tel223-api
```



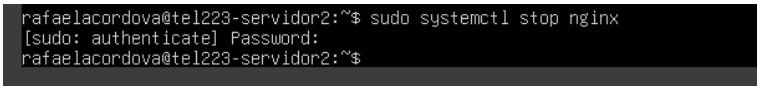




# ETAPA 6

## Falla 1: Servidor web caído - Falla 3

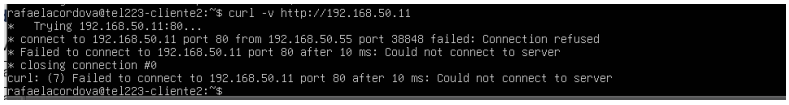
- **1. Provocar falla (En la VM SERVIDOR):**

  `sudo systemctl stop nginx`




- **2. Generar evidencia (En la VM CLIENTE):**

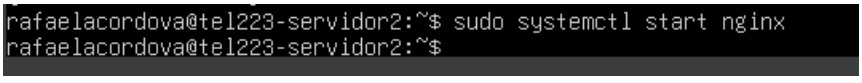
  `curl -v http://192.168.50.11`




  *Lo que debes ver en tu captura:* La terminal arrojará un error rápido indicando `Failed to connect to 192.168.50.11 port 80: Connection refused`.

- **3. Restaurar el servicio (En la VM SERVIDOR):**

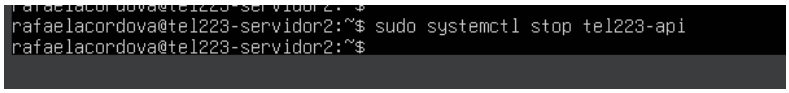
  `sudo systemctl start nginx`



### Falla 2: Backend de la API detenido (Corresponde a F4 de tu tabla)

- **1. Provocar falla (En la VM SERVIDOR):**

  `sudo systemctl stop tel223-api`




- **2. Generar evidencia desde fuera (En la VM CLIENTE):**

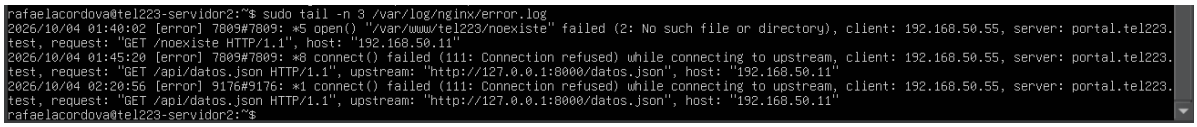
  `curl -i http://192.168.50.11/api/datos.json`




  *Lo que debes ver en tu captura:* Nginx seguirá respondiendo, pero mostrará un error `HTTP/1.1 502 Bad Gateway`.

- **3. Generar evidencia interna (En la VM SERVIDOR):**

  `sudo tail -n 3 /var/log/nginx/error.log`




  *Lo que debes ver en tu captura:* El registro detallado que confiesa el problema: `connect() failed (111: Connection refused) while connecting to upstream`.

- **4. Restaurar el servicio (En la VM SERVIDOR):**

  `sudo systemctl start tel223-api`

